# Mitigating Unreliable Ranking for Heavy Hitter Identification in Large Domains under Local Differential Privacy

Yuanbo Zhao
Southeast University
Nanjing, China
zhaoyb@seu.edu.cn

Kai Dong*
Southeast University
Nanjing, China
dk@seu.edu.cn

Mengxiang Quan
Southeast University
Nanjing, China
mengxiang_quan@seu.edu.cn

Huanhuan Jia
Southeast University
Nanjing, China
hhjia@seu.edu.cn

Beibei Cheng
Southeast University
Nanjing, China
bbcheng@seu.edu.cn

Zhen Ling
Southeast University
Nanjing, China
zhenling@seu.edu.cn

Xinwen Fu
University of
Massachusetts Lowell
Lowell, United States
xinwen_fu@uml.edu

## ABSTRACT

Heavy hitter identification (HHI) under local differential privacy (LDP) is a critical problem in distributed data analysis and is commonly addressed by ranking noisy frequency estimates obtained from frequency estimation mechanisms known as frequency oracles (FOs). While widely used FOs can provide unbiased frequency estimates for any given value with estimation variance independent of the domain size, HHI accuracy still degrades in large domains. We refer to this challenge as *unreliable ranking*. To address this challenge, we propose the BSS-strategy, which avoids directly ranking noisy frequency estimates over the entire domain and instead reconstructs candidates from bit-level statistics using a Bit-Support-Set representation. We further extend the BSS-strategy with a conditional expansion phase that partitions the domain using selected bits and reconstructs candidates more accurately within each partition using conditional bit statistics. Building on these ideas, we design BSS-HHI, a multi-phase protocol for large-domain HHI under LDP. We theoretically prove that BSS-HHI satisfies $\varepsilon$-LDP and provide a theoretical analysis of its utility and overhead. Extensive experiments on three real-world and two synthetic datasets demonstrate that BSS-HHI significantly outperforms state-of-the-art baselines in large-domain settings, achieving 10% $\sim$ 30% improvements in accuracy. Furthermore, we evaluate the proposed BSS-based methodology in two case studies on federated HHI and relation mining.

**PVLDB Reference Format:**
Yuanbo Zhao, Kai Dong, Mengxiang Quan, Huanhuan Jia, Beibei Cheng, Zhen Ling, and Xinwen Fu. Mitigating Unreliable Ranking for HHI under LDP. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://github.com/zybxav/BSS-HHI.

*Corresponding Author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

## 1 INTRODUCTION

Heavy hitter identification (HHI) for private data in distributed settings is a critical problem [30, 42, 43, 56], and is widely applied in scenarios such as hotspot event monitoring [49] and web analysis [36]. It aims to identify the most frequent values, namely heavy hitters, from a value domain based on data held by individual users, without compromising individual privacy. Given that local differential privacy (LDP) [29] has become a widely adopted standard for protecting user privacy in distributed settings, HHI protocols under LDP have been extensively studied [4, 5, 8, 18, 30, 33, 37, 42, 43, 51, 56].

To the best of our knowledge, all existing HHI protocols rely on LDP frequency estimation mechanisms, known as **frequency oracles (FOs)** [41]. An FO consists of a pair of algorithms: a local perturbation algorithm executed by each user to perturb its true value, and an aggregation algorithm run by the server to estimate value frequencies from the perturbed reports. For a given value, widely used FOs (e.g., OLH, SUE and OUE) can provide unbiased frequency estimates, and their estimation variance can be made independent of the domain size [41]. Therefore, a natural way to perform HHI with FOs is to estimate the frequencies of all values in the domain and then rank them to identify the heavy hitters.

However, the accuracy of FO-based HHI protocols degrades significantly when the domain becomes excessively large [5]. The core reason is that FO provides domain-independent estimates for any given value, whereas HHI requires a domain-dependent ranking process on the estimates. As the domain size grows, although the scale of LDP noise remains unchanged, the average frequency gaps among these values become much smaller. When these gaps fall below the noise level, the ranking of FO-based estimates becomes unreliable. We refer to this challenge as **unreliable ranking**. In Section 3.2, we illustrate this challenge using datasets with varying domain sizes under the same underlying distribution, and evaluate both top-16 and top-1% HHI. As the domain size increases from $2^7$ to $2^{17}$, the average accuracy drops from 85% to 45% in the top-16 setting, and from 100% to 2% in the top-1% setting.

To avoid directly ranking FO-based estimates over the entire domain, some recent methods adopt grouping-based strategies [8,

37, 43, 51, 56]. For example, prefix-tree-based methods group values by shared prefixes, rank these groups by their estimated frequencies, prune low-frequency groups, and progressively expand the remaining prefixes. These strategies perform ranking at the group level rather than over the entire domain, which can improve accuracy. However, prefix-tree-based grouping starts from the beginning of the domain encoding and progressively extends prefixes using consecutive bits. Frequent patterns in later or nonconsecutive bits cannot be identified in early rounds, so the preceding prefixes must be expanded before these patterns can be reached. This requires repeated ranking over multiple rounds under LDP noise, with early ranking errors becoming irreversible once the corresponding groups are pruned. Thus, their progressive search remains affected by unreliable ranking.

To address this challenge, we adopt a candidate reconstruction strategy that avoids directly ranking noisy frequency estimates over the entire domain. It reconstructs candidates from their binary representation of length $l$ using a specially designed data structure, which we term a Bit-Support-Set (BSS), where for each value, its BSS is defined as the set of indices of bits whose values are equal to 1 in its binary representation. For example, when $l = 4$, the BSS of a value $v = 2$ (binary 0010) is $\{3\}$, as the only bit set to 1 is at position 3. From the BSS of values, we estimate the probability of each bit position under LDP, and combine these bit-level probabilities to reconstruct candidate values. We refer to this candidate reconstruction strategy as **BSS-strategy**. For a value $v$ with binary representation $(b_1(v), \ldots, b_l(v))$, its reconstruction score combines its estimated marginal bit probabilities as $\mathcal{P}(v) = \prod_{j=1}^{l} \hat{p}_{j,b_j(v)}$, where $\hat{p}_{j,b_j(v)}$ denotes the estimated probability of the $j$-th bit value. This formulation allows high-probability candidates to be reconstructed directly by combining the most likely bit patterns. Such reconstruction captures the statistical characteristics of heavy hitters at the bit level, as a heavy hitter can be formed by combining bit values with high individual probabilities. By avoiding explicit ranking over noisy frequency estimates, this BSS-strategy generates a more reliable candidate domain.

We further extend our BSS-strategy with a **conditional expansion** phase, which reconstructs candidates in a more fine-grained manner by partitioning the domain. This phase first selects one or several bits whose estimated probabilities are most imbalanced, measured by their deviation from 0.5. For each selected bit, the domain is divided into two partitions according to its bit value. For example, when $l = 4$ and the first bit position is selected, the domain is divided into two partitions corresponding to values with $b_1(v) = 0$ and $b_1(v) = 1$. Within each partition, we estimate the conditional probabilities of the remaining bits, such as $\Pr(b_j(v) = 1 \mid b_1(v) = 0)$ and $\Pr(b_j(v) = 1 \mid b_1(v) = 1)$ for $j = 2, 3, 4$, and reconstruct candidates accordingly. This process can be applied iteratively within each partition by selecting additional bit positions and estimating conditional probabilities given more known bit values. This phase mitigates the dominant influence of highly imbalanced bits in candidate reconstruction. Such bits can overwhelm the contribution of other bits to the reconstruction score, letting the higher-probability bit value dominate candidate scores. By reconstructing candidates separately within each partition, conditional expansion mitigates this dominance and yields a more accurate candidate domain.

Building upon our BSS-strategy and conditional expansion phase, we further propose BSS-HHI (BSS-based Heavy Hitter Identifier), a multi-phase HHI protocol that achieves higher accuracy in large domains under LDP. In addition to these two phases, BSS-HHI further incorporates an SVD filter phase and a final HHI phase. The SVD filter phase enables the final HHI to operate on a smaller and more refined candidate domain. It leverages matrix factorization with low-rank approximation, inspired by LDP-RM [11], and combines it with an $\ell_2$-ball mechanism [13] to estimate the matrix under LDP and refine the candidate domain.

We conduct rigorous theoretical analysis and empirical evaluation to validate BSS-HHI. We formally prove that BSS-HHI satisfies $\varepsilon$-LDP and analyze its utility and overhead. We conduct comprehensive experiments on three real-world and two synthetic datasets to evaluate the performance of the proposed BSS-HHI protocol, comparing it against mainstream baselines. The results demonstrate that BSS-HHI holds a significant advantage on datasets with domain sizes exceeding 40k, achieving a 10% $\sim$ 30% improvement in accuracy. We further evaluate BSS-based candidate reconstruction through case studies on federated HHI and relation mining. We also conduct parameter tuning to evaluate the sensitivity and effectiveness of key components in BSS-HHI.

Our contributions are summarized as follows:

(1) **New Methodology.** We propose the BSS-strategy, a new candidate reconstruction method that builds candidates from bit-level statistics rather than directly ranking noisy frequency estimates over the entire domain. We further extend the BSS-strategy with a conditional expansion phase via conditional estimation to more accurately reconstruct candidates. Building on these ideas, we propose BSS-HHI, a multi-phase HHI protocol that integrates the BSS-strategy, conditional expansion, an SVD filter phase, and a final HHI phase to accurately identify heavy hitters in large domains under LDP.

(2) **Theoretical and Experimental Analysis.** We provide rigorous theoretical analysis, proving that BSS-HHI satisfies $\varepsilon$-LDP and characterizing its utility, communication, and computation overhead. Extensive experiments on three real-world and two synthetic datasets demonstrate that BSS-HHI consistently outperforms state-of-the-art baselines in large-domain settings. We also present relation mining and federated HHI as two evaluated case studies of BSS-based candidate reconstruction.

## 2 PRELIMINARIES

This section introduces local differential privacy, frequency oracles, and the set-valued item mining [42] protocol.

### 2.1 Local Differential Privacy

Local differential privacy (LDP) [29] is a variant of differential privacy (DP) designed for decentralized settings, offering strong privacy guarantees without requiring a trusted aggregator. In the decentralized settings, each user applies a randomized algorithm $\Psi$ to their private value $x \in X$ and reports the perturbed result $\Psi(x)$. To ensure privacy, the algorithm $\Psi$ should satisfy:

*Definition 2.1 ($\varepsilon$-Local Differential Privacy).* An algorithm $\Psi(\cdot)$ satisfies $\varepsilon$-local differential privacy ($\varepsilon$-LDP), where $\varepsilon \geq 0$, if and

only if for any two values $x_1, x_2 \in X$, we have:

$$\forall T \in Range(\Psi) : \Pr[\Psi(x_1) = T] \leq e^{\varepsilon} \Pr[\Psi(x_2) = T],$$

where $Range(\Psi)$ denotes the range of $\Psi$.

Here, $\varepsilon$ quantifies the trade-off between privacy and utility in the algorithm and is commonly referred to as the *privacy budget*. A smaller value of $\varepsilon$ offers stronger privacy guarantees, but at the expense of reduced utility. When a user needs to report multiple values, the *sequential composition theorem* [14] requires the total privacy budget to be divided across reports. This division results in less budget allocated per value, leading to excessive perturbation and reduced estimation accuracy. To mitigate this, users are typically partitioned into disjoint groups, each reporting a single value, thereby preserving utility while still satisfying LDP.

## 2.2 Frequency Oracles

A frequency oracle (FO) is an LDP protocol designed to estimate the frequency of any given value $x \in X$. An FO is typically defined as a pair of algorithms $\mathcal{M}(\varepsilon) = \langle \Psi_{\mathcal{M}(\varepsilon)}, \Phi_{\mathcal{M}(\varepsilon)} \rangle$, where $\Psi_{\mathcal{M}(\varepsilon)}$ is the user-side perturbation algorithm satisfying $\varepsilon$-LDP, and $\Phi_{\mathcal{M}(\varepsilon)}$ is the server-side aggregation function. Given privatized reports, we use $\Phi_{\mathcal{M}(\varepsilon)}(x)$ to denote the estimated frequency of value $x$.

In practice, commonly used FOs include generalized randomized response (GRR) [44], optimized local hashing (OLH), optimized unary encoding (OUE), and basic RAPPOR (SUE) [41]. Unlike GRR, the estimation error of OLH, OUE, and SUE is independent of the domain size.

## 2.3 SVIM

SVIM [42] is a representative LDP protocol for identifying frequent values when each user holds a private value set $\mathbf{x}_i \subseteq X$. It builds upon a padding-and-sampling-based frequency oracle (PSFO) [33], which extends standard single-value FOs to set-valued inputs. Given a value set $\mathbf{x}_i$ and a padding length $\ell$, the padding-and-sampling function $\mathrm{PS}_\ell(\mathbf{x}_i)$ pads $\mathbf{x}_i$ with dummy values if $|\mathbf{x}_i| < \ell$, and then uniformly samples one value from the resulting set. The sampled value is then perturbed using an underlying FO, and the server aggregates the perturbed reports to estimate item frequencies.

SVIM uses different user groups to complete three estimation tasks. First, one group reports sampled values through PSFO with padding length $\ell = 1$, and the server selects the top-$2k$ estimated values as a candidate set $S$. Second, another group reports the intersection size $t_i = |\mathbf{x}_i \cap S|$ using OLH. For each $t \in \{0, 1, \ldots, 2k\}$, let $\hat{q}_t = \Phi_{\mathrm{OLH}(\varepsilon)}(t)$ denote the estimated fraction of users with $t_i = t$. The server selects the 90-th percentile of this estimated distribution as the padding length $L$. Third, the remaining group applies $\mathrm{PS}_L$ to $\mathbf{x}_i \cap S$ and reports the sampled value through PSFO. The server estimates the frequencies of values in $S$, applies the correction step of SVIM to compensate for the bias introduced by padding and sampling, and outputs the top-$k$ values with their estimated frequencies. We denote the server-side aggregation output of SVIM by

$$\Phi_{\mathrm{SVIM}}(\{\mathbf{x}_i\}_{i=1}^n; \varepsilon, k) = \{(x, \hat{f}_x) \mid x \in X_k\}, \tag{1}$$

where $X_k$ is the set of $k$ values selected by SVIM, and $\hat{f}_x$ is the estimated frequency of value $x$.

# 3 PROBLEM AND CHALLENGE

In this section, we discuss the problem of heavy hitter identification (HHI) under LDP in large domains and propose a new research challenge based on our observations.

## 3.1 Problem: HHI in Large Domains

The problem of *top-k heavy hitter identification* (HHI) under LDP has been studied in prior works such as [42, 43, 56], whereas our work focuses specifically on HHI under LDP in large domains. We consider a setting with an untrustworthy aggregator, a set of $n$ distributed users $U = \{u_1, u_2, \ldots, u_n\}$, and a global value domain $X = \{v_1, v_2, \ldots, v_d\}$, where $d = |\mathcal{X}|$ denotes the size of the domain. Each user $u_i \in U$ holds a private subset of values $\mathbf{x}_i \subseteq \mathcal{X}$.

Given the global value domain $\mathcal{X}$, the frequency of value $v \in \mathcal{X}$ is defined as

$$f(v) = \frac{\sum_{i=1}^n \mathbb{I}[v \in \mathbf{x}_i]}{n}, \tag{2}$$

where $\mathbb{I}[\cdot]$ is the indicator function. Since each user contributes at most one occurrence to any given value, $f(v)$ can be interpreted as the fraction of users whose local sets contain $v$. The notion of top-$k$ heavy hitters is defined as follows:

*Definition 3.1 (Top-k Heavy Hitters).* The top-$k$ heavy hitters are defined as the set of $k$ values with the highest frequencies:

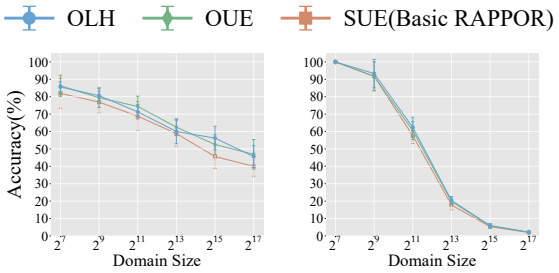
$$H_k = \{v^{(1)}, v^{(2)}, \ldots, v^{(k)}\} \subseteq \mathcal{X},$$

where $f(v^{(1)}) \geq f(v^{(2)}) \geq \cdots \geq f(v^{(k)})$, and for any $v \in \mathcal{X} \setminus H_k$, we have $f(v) \leq f(v^{(k)})$.

## 3.2 Challenge: Unreliable Ranking

When the domain becomes excessively large, the accuracy of FO-based HHI protocols degrades significantly. Prior work studies HHI from the perspective of succinct histograms, which output the most frequent values together with their frequency estimates, and shows that the error of identifying these values depends on the domain size [5]. Under this view, the core reason is that FO provides domain-independent estimates for any given value, whereas HHI relies on a domain-dependent ranking process over these estimates. As the domain size increases, the normalized frequency gaps among values become smaller. Although the scale of LDP noise remains unchanged for each value, such noise can easily exceed these small frequency gaps and make the ranking unreliable. We refer to this challenge as **unreliable ranking**.

To illustrate this challenge, we construct synthetic datasets following a Zipf distribution, where the frequency of the $i$-th value is proportional to $i^{-1.5}$. We fix the number of users and progressively increase the domain size. We then apply representative FOs, including OLH, OUE, and Basic RAPPOR (SUE, RAPPOR [16] without repeated reporting) [41], to obtain frequency estimates, and rank these estimates to identify heavy hitters. As shown in Figure 1, under both the top-16 and top-1% settings, the accuracies of all FOs decrease as the domain size increases under a fixed privacy budget. In the top-16 setting, the accuracy drops substantially, from around 85% to about 45% when the domain reaches $2^{17}$. The degradation is even more pronounced in the top-1% setting, where the accuracy decreases sharply from 100% to around 2% as the domain grows.




**(a) Accuracy of top-16 HHI (b) Accuracy of top-1% HHI**

**Figure 1: Identification accuracy of heavy hitters by ranking FO-based estimates under the same distribution with varying domain sizes.**

Some recent methods for large domains adopt grouping-based strategies, which are often implemented through prefix-tree-based mechanisms [8, 37, 43, 51, 56]. Such methods encode values as fixed-length bit or character strings and partition the encoded domain according to prefixes. They first identify short prefixes that appear to be frequent by ranking their estimated frequencies. These prefixes are then progressively expanded to longer ones, retaining those that remain frequent based on ranking results, until they reach the full length. These strategies perform ranking at the prefix level rather than over the entire domain, thereby improving accuracy. However, prefix-tree-based grouping starts from the beginning of the domain encoding and progressively extends prefixes using consecutive bits. When frequent patterns appear in later or nonconsecutive bits, they cannot be identified directly at early levels. Instead, the prefixes preceding these patterns must first be explored and progressively expanded, potentially leaving multiple groups to be considered before the relevant patterns are reached. At each round, these groups are ranked according to their FO-based frequency estimates to determine which ones are retained for further expansion. The ranking is therefore repeatedly performed before the relevant patterns can be identified. Moreover, an error made in an early round can affect all subsequent exploration: once a truly frequent prefix is mistakenly pruned, its subsequent prefixes are no longer explored and cannot be recovered in later stages. Consequently, the progressive search remains affected by unreliable ranking.

# 4 METHOD

In this section, we present BSS-HHI (short for *Bit-Support-Set-based Heavy Hitter Identifier*) and describe how it addresses the challenge.

## 4.1 Overview

BSS-HHI is a novel HHI protocol under LDP designed for large domains. It operates in four phases: a *bit-support-set (*BSS*)-strategy* phase, a *conditional expansion* phase, an *SVD filter* phase, and a *final HHI* phase via SVIM. The BSS-*strategy* phase leverages bit-level patterns to reconstruct candidate values from a specially designed structure, termed the B*it-S*upport-*S*et (BSS). The *conditional expansion* phase further incorporates conditional bit probability estimation to more accurately reconstruct candidates. The *SVD filter* phase then exploits latent structural patterns in the data to achieve a higher-quality pruning. These three phases progressively narrow down the original domain to a focused candidate set, in which the *final HHI* phase identifies the top-$k$ heavy hitters. To save privacy budget, users are randomly partitioned into three disjoint groups $(\mathcal{U}_1, \mathcal{U}_2, \mathcal{U}_3)$, where $\mathcal{U}_1$ is used for the tasks in Phases 1 and 2, $\mathcal{U}_2$ for Phase 3, and $\mathcal{U}_3$ for Phase 4; within each group, users are further partitioned into disjoint subgroups assigned to different tasks, so that each user reports exactly once. The workflow of BSS-HHI is illustrated in Fig. 2.

## 4.2 Phase 1: BSS-strategy

In this phase, the BSS-strategy aims to reconstruct candidate values by exploiting bit-level probability information, rather than directly ranking noisy frequency estimates over the entire domain. It consists of three steps: bit support set construction, bit probability estimation, and candidate value reconstruction. In the first step, the BSS-strategy introduces a representation called the Bit-Support-Set (BSS), which represents each value by the positions of its bits with value one. Based on this representation, in the second step, the protocol estimates the probability of observing bit value one at each position. By decomposing a value into multiple bit dimensions, this approach captures probability information at the level of individual bits rather than relying on a single value-level estimate. This multi-dimensional representation describes each value in terms of its constituent bits, enabling more fine-grained probability modeling. In the third step, the BSS-strategy reconstructs candidate values by combining high-probability bit values across positions. Since the binary encoding decomposes a value into multiple bit dimensions, the likelihood of a value can be characterized by the joint occurrence probability of its constituent bits. Values formed by combining bit values that individually have high probabilities are therefore more likely to appear in the data, yielding a focused candidate domain.

***Step 1.1: Bit Support Set Construction.*** In this step, each user assigned to this step from the first user-group ($u \in \mathcal{U}_1$) randomly samples a value $v_u$ from her private set $\mathbf{x}_u$ using the sampling function $\mathrm{PS}_{\ell=1}$, given by:

$$v_u = \mathrm{PS}_{\ell=1}(\mathbf{x}_u). \tag{3}$$

This sampled value $v_u$ is encoded as a binary string of length $l = \lceil \log_2 d \rceil$, where $d = |\mathcal{X}|$ denotes the domain size. Then, $v_u$ is represented by its bit support set $\mathrm{BSS}(v_u)$, defined as follows:

*Definition 4.1 (*Bit-Support-Set (BSS)*).* Let $v_u \in \mathcal{X}$ be a value whose binary encoding has length $l$. The Bit-Support-Set (BSS) of $v_u$ is defined as the index set of bits whose values are 1 in its binary encoding:

$$\mathrm{BSS}(v_u) = \{j \in [l] \mid b_j(v_u) = 1\},$$

where $b_j(v_u)$ denotes the $j$-th bit of the binary encoding of $v_u$.

For example, consider a dataset with domain size $d = 2^4$, whose values are encoded from 0000 to 1111. If a user samples 0111 from her private set, then $\mathrm{BSS}(0111) = \{2, 3, 4\}$. The resulting $\mathrm{BSS}(v_u)$ thus serves as the BSS of user $u$ and will be used in subsequent steps.

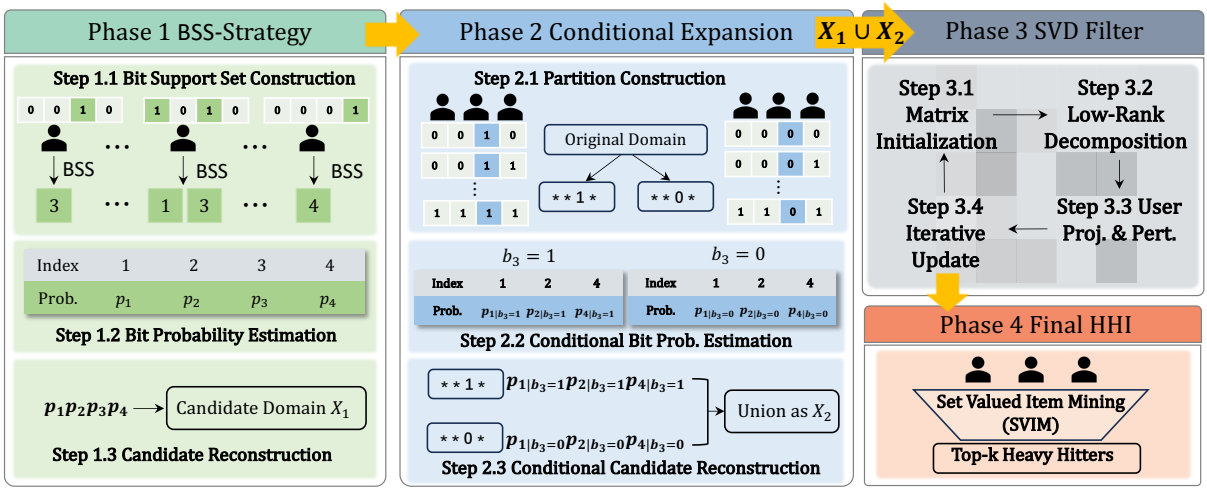



**Figure 2: Workflow of BSS-HHI. Phase 1 (BSS-strategy), performed by User Group 1 and the Aggregator, reconstructs candidate domain $X_1$. Phase 2 (Conditional Expansion), performed by User Group 1 and the Aggregator, reconstructs candidate domain $X_2$. Phase 3 (SVD Filter), performed by User Group 2 and the Aggregator, refines the candidate domain. Phase 4 (Final HHI), performed by User Group 3 and the Aggregator, identifies the top-$k$ heavy hitters from the refined domain using SVIM.**

***Step 1.2: Bit Probability Estimation.*** In this step, the aggregator and users assigned to this step from $\mathcal{U}_1$ employ an SVIM protocol to estimate the probability of each $j \in [l]$ based on BSS representations. Let $\hat{p}_{j,1}$ denote the estimated probability that bit $j$ equals 1 across the participating users, and let $\hat{p}_{j,0} = 1 - \hat{p}_{j,1}$. Formally, the estimation is given by:

$$\Phi_{\text{SVIM}}\big(\{\text{BSS}(v_u)\}_{u\in\mathcal{U}_1}; \varepsilon, k = l\big) = \{(j, \hat{p}_{j,1}) \mid j \in [l]\}, \tag{4}$$

For example, $\{2, 3, 4\}$ is used as a user's set-valued input to SVIM, and an output $\hat{p}_{2,1} = 0.6$ means that the second bit is estimated to equal 1 with probability 0.6.

***Step 1.3: Candidate Reconstruction.*** In this step, the aggregator computes the reconstruction scores $\mathcal{P}$ of each possible value $v \in \mathcal{X}$ from the estimated probabilities $\hat{p}_{i,b_i(v)}$:

$$\mathcal{P}(v) = \prod_{i=1}^{l} \hat{p}_{i,b_i(v)}. \tag{5}$$

For example, when $v = 0111$, its reconstruction score is $\mathcal{P}(0111) = \hat{p}_{1,0}\hat{p}_{2,1}\hat{p}_{3,1}\hat{p}_{4,1}$. The aggregator then selects the top values according to the reconstruction scores, which form the candidate domain $X_1$.

***Formalization.*** Phase 1 can be formalized as a pair of algorithms $\langle \Psi_{\text{BSS}_1(|\mathcal{U}_1|,\varepsilon)}, \Phi_{\text{BSS}_1(|\mathcal{U}_1|,\varepsilon)} \rangle$:

$$y_u = \Psi_{\text{BSS}_1(|\mathcal{U}_1|,\varepsilon)}(\mathbf{x}_u) \triangleq \Psi_{\text{SVIM}(\varepsilon)}(\text{BSS}(v_u)), \tag{6}$$

$$\Phi_{\text{BSS}_1(|\mathcal{U}_1|,\varepsilon)}\big(\{y_u\}_{u\in\mathcal{U}_1}\big) \triangleq X_1. \tag{7}$$

## 4.3 Phase 2: Conditional Expansion

In this phase, conditional expansion extends the BSS-strategy by incorporating conditional bit probability statistics to reconstruct candidate values more accurately. It consists of three steps: partition construction, conditional bit probability estimation, and conditional candidate reconstruction. In the first step, the aggregator selects several bits whose estimated probabilities deviate most from 0.5, and uses each selected bit to partition the domain into two parts according to whether this bit equals 0 or 1. In the second step, within each partition, the protocol estimates the conditional probabilities of the remaining bits given the selected bit values. In the third step, the aggregator reconstructs a candidate set within each partition using these conditional bit probabilities. These steps can be applied iteratively within each partition by selecting additional bits, leading to candidate reconstruction under progressively more specific bit patterns. This design is motivated by the fact that bits with probabilities far from 0.5 can dominate the reconstruction score. Since the BSS-strategy reconstructs a value by multiplying bit-level probabilities, highly imbalanced bits can overwhelm the contribution of other bits to the reconstruction score. As a result, the higher-probability bit value can dominate the reconstruction scores. By partitioning the domain according to these bits and estimating the conditional probabilities of the remaining bits within each partition, conditional expansion mitigates this dominance and reconstructs candidates more accurately.

***Step 2.1: Partition Construction.*** In this step, the aggregator selects the most imbalanced bits, i.e., those whose estimated probabilities deviate most from 0.5, and uses each selected bit independently to partition the domain according to its bit value. A partition is represented by a bit pattern $\tau = (\tau_1, \ldots, \tau_l)$, where $\tau_j \in \{0, 1, *\}$. Here, $\tau_j = 0$ or $\tau_j = 1$ means that the $j$-th bit is specified to be 0 or 1, while $\tau_j = *$ means that it is not specified. The partition induced by $\tau$ is defined as

$$\mathcal{X}^{(\tau)} = \big\{v \in \mathcal{X} \mid b_j(v) = \tau_j,\ \forall j \text{ such that } \tau_j \in \{0, 1\}\big\}, \tag{8}$$

where $b_j(v)$ denotes the $j$-th bit of value $v$.

For example, with $d = 2^4$, selecting bit 1 partitions the domain into $0***$ (0000-0111) and $1***$ (1000-1111). If bit 3 is also selected

in the same iteration, it independently partitions the domain into $**0*$ and $**1*$.

***Step 2.2: Conditional Bit Probability Estimation.*** In this step, the aggregator and users assigned to this step from $\mathcal{U}_1$ estimate conditional bit probabilities within each partition. For a partition $\mathcal{X}^{(\tau)}$ and any bit index $j$ with $\tau_j = *$, the conditional bit probabilities are defined as

$$p_{j,1}^{(\tau)} = \Pr\Big(b_j(v_u) = 1 \mid v_u \in \mathcal{X}^{(\tau)}\Big). \tag{9}$$

Following Steps 1.1 and 1.2, the BSS representations are constructed over the bit indices $j$ with $\tau_j = *$, and SVIM is applied to estimate the conditional bit probabilities for the current partition. For example, if $\tau = (0, *, *, *)$, then

$$p_{3,1}^{(0***)} = \Pr(b_3(v_u) = 1 \mid b_1(v_u) = 0). \tag{10}$$

This yields the estimates $\hat{p}_{j,1}^{(\tau)}$ and $\hat{p}_{j,0}^{(\tau)} = 1 - \hat{p}_{j,1}^{(\tau)}$. These estimates characterize the conditional bit statistics given the selected bits.

***Step 2.3: Conditional Candidate Reconstruction.*** In this step, the aggregator reconstructs candidates separately within each partition using the corresponding conditional bit probabilities. Following Step 1.3, for a value $v \in \mathcal{X}^{(\tau)}$, its score in partition $\mathcal{X}^{(\tau)}$ is defined as

$$\mathcal{P}^{(\tau)}(v) = \prod_{j:\tau_j=*} \hat{p}_{j,b_j(v)}^{(\tau)}. \tag{11}$$

The selected bits do not contribute to the conditional reconstruction score. For each partition $\mathcal{X}^{(\tau)}$, the aggregator selects a set of values with the largest reconstruction scores, forming a candidate set $\widehat{X}^{(\tau)}$ for this partition. The output of this phase is the union of the candidates reconstructed from all partitions:

$$X_2 = \bigcup_{\tau \in \mathcal{T}} \widehat{X}^{(\tau)}, \tag{12}$$

where $\mathcal{T}$ denotes the collection of partition patterns generated during this phase.

When these steps are applied iteratively, an additional selected bit further specifies an existing partition. For example, after constructing $1 * **$, selecting bit 3 in a subsequent iteration constructs $1 * 0*$ and $1 * 1*$.

***Formalization.*** Phase 2 can be formalized as a pair of algorithms $\langle \Psi_{\mathrm{CE}(|\mathcal{U}_1|,\varepsilon)}, \Phi_{\mathrm{CE}(|\mathcal{U}_1|,\varepsilon)} \rangle$:

$$y_u^{(\tau)} = \Psi_{\mathrm{CE}(|\mathcal{U}_1|,\varepsilon)}^{(\tau)}(\mathbf{x}_u) \triangleq \Psi_{\mathrm{SVIM}(\varepsilon)}\big(\mathsf{BSS}^{(\tau)}(v_u)\big), \tag{13}$$

$$\Phi_{\mathrm{CE}(|\mathcal{U}_1|,\varepsilon)}\left(\left\{\{y_u^{(\tau)}\}_{v_u \in \mathcal{X}^{(\tau)}}\right\}_{\tau \in \mathcal{T}}\right) \triangleq \bigcup_{\tau \in \mathcal{T}} \widehat{X}^{(\tau)} = X_2. \tag{14}$$

Here,

$$\mathsf{BSS}^{(\tau)}(v_u) = \{j \in [l] \mid \tau_j = *,\ b_j(v_u) = 1\}, \quad v_u \in \mathcal{X}^{(\tau)}. \tag{15}$$

## 4.4 Phase 3: SVD Filter

The SVD filter phase is designed to further refine the candidate domain obtained in Phases 1 and 2. It consists of four steps: matrix initialization, low-rank decomposition, user projection and perturbation, and matrix updating. Inspired by LDP-RM [11], an LDP relation mining protocol, we represent frequency estimates in a matrix form and iteratively refine this matrix through singular value decomposition and low-rank approximation. This process preserves dominant patterns that capture high-frequency structures, thereby further reducing the number of values that require final estimation. However, in large domains, collecting and perturbing the resulting high-dimensional statistics can still introduce substantial noise. To address this issue, we employ the $\ell_2$-ball mechanism [13], which is well suited for perturbing high-dimensional data. As a result, this phase enables further domain refinement without requiring exact frequency estimation for all remaining values.

***Step 3.1: Matrix Initialization.*** In this step, the aggregator first forms the candidate domain as the union of the outputs from the previous phases, denoted by $X_1 \cup X_2$. To initialize the frequency matrix in a unified manner, each value $v \in X_1 \cup X_2$ is assigned its Phase 1 reconstruction score $\mathcal{P}(v)$. The aggregator sorts the candidates in descending order of $\mathcal{P}(v)$ and fills their scores into an $s \times s$ matrix $M_{\mathrm{init}} \in \mathbb{R}^{s \times s}$ row by row, where $s = \lceil \sqrt{|X_1 \cup X_2|} \rceil$. Consecutive rows contain adjacent segments of the decreasing score sequence and therefore tend to have similar profiles, providing a low-rank structure for approximation. For example, five ordered scores $q_1 \geq \cdots \geq q_5$ are placed row by row as $M_{\mathrm{init}} = [q_1, q_2, q_3;\ q_4, q_5, 0;\ 0, 0, 0]$. Each entry of the matrix corresponds to one value in $X_1 \cup X_2$ and is initialized with its score $\mathcal{P}(v)$. If the matrix has more entries than the number of candidate values, the remaining entries are filled with zeros.

For each user $u \in \mathcal{U}_2$, a binary matrix $I_u \in \{0,1\}^{s \times s}$ is constructed using the same descending-score, row-by-row ordering as in $M_{\mathrm{init}}$. Each entry corresponds to a value in $X_1 \cup X_2$, and is set to 1 if that value appears in the user's private set $\mathbf{x}_u$, and 0 otherwise.

***Step 3.2: Low-Rank Decomposition.*** In this step, the aggregator performs singular value decomposition on $M_{\mathrm{init}}$:

$$M_{\mathrm{init}} = G_s \Sigma_s H_s^\top. \tag{16}$$

It then constructs a low-rank approximation by retaining the top $r$ singular values and their corresponding singular vectors:

$$M_r = G_r \Sigma_r H_r^\top = \sum_{i=1}^{r} \sigma_i g_i h_i^\top. \tag{17}$$

Here, $G_r$ and $H_r$ contain the top $r$ left and right singular vectors, and $\Sigma_r$ contains the corresponding singular values. The rank $r$ is chosen as the smallest integer such that the cumulative singular-value mass $\sum_{i=1}^{r} \sigma_i / \sum_i \sigma_i$ exceeds a threshold $\theta$.

***Step 3.3: User Projection and Perturbation.*** In this step, the aggregator sends $G_r$ and $H_r$ to users in $\mathcal{U}_2$. Each user projects the local binary matrix $I_u$ into the reduced subspace:

$$M_u = G_r^\top I_u H_r. \tag{18}$$

Let $Y_u = \mathrm{vec}(M_u) \in \mathbb{R}^{r^2}$ denote the vectorized form of $M_u$. Note that $M_u$ is generally not diagonal, and its off-diagonal entries cannot be ignored, since they also contain useful structural information. Therefore, to obtain a more accurate estimate, the aggregator needs to collect users' reports over the entire matrix $M_u$ rather than only its diagonal entries. However, this makes the local report high-dimensional, and perturbing all entries separately would introduce substantial noise under LDP. To address this issue, we employ the $\ell_2$-ball mechanism [13], which perturbs the whole vector jointly. Each user first clips $Y_u$ to a fixed radius $c$, and then maps it to

a point $\tilde{Y}_u$ on the sphere of radius $c$. Let $q_\varepsilon = \frac{e^\varepsilon}{e^\varepsilon+1}$, and sample $Z_u \sim \text{Bernoulli}(q_\varepsilon)$. The sampling radius is set as

$$B = c \cdot \frac{e^\varepsilon + 1}{e^\varepsilon - 1} \cdot \frac{r^2\sqrt{\pi}\,\Gamma\left(\frac{r^2+1}{2}\right)}{2\,\Gamma\left(\frac{r^2}{2}+1\right)}. \tag{19}$$

The perturbed vector $Y'_u$ is then drawn uniformly from the sphere of radius $B$ in the hemisphere determined by $\tilde{Y}_u$:

$$Y'_u \sim \begin{cases} \text{Unif}\left(\left\{z \in \mathbb{R}^{r^2} \mid \langle z, \tilde{Y}_u\rangle > 0,\ \|z\|_2 = B\right\}\right), & \text{if } Z_u = 1, \\ \text{Unif}\left(\left\{z \in \mathbb{R}^{r^2} \mid \langle z, \tilde{Y}_u\rangle \le 0,\ \|z\|_2 = B\right\}\right), & \text{if } Z_u = 0. \end{cases} \tag{20}$$

This avoids allocating the privacy budget independently to each dimension and is more suitable for the high-dimensional report $Y_u$.

***Step 3.4: Iterative Update.*** In this step, the aggregator averages the perturbed user vectors, reshapes the result into a matrix $\hat{M}_r^{(t)} \in \mathbb{R}^{r\times r}$, and updates the frequency matrix by

$$M_{\text{init}}^{(t)} = G_r \hat{M}_r^{(t)} H_r^\top. \tag{21}$$

This phase can be repeated for $T$ rounds by partitioning $\mathcal{U}_2$ into $T$ disjoint subgroups, so that each user participates in exactly one round. After the final round, the aggregator obtains the refined matrix and selects the values corresponding to the top entries as the new candidate domain $X_3$.

***Formalization.*** This phase can be formalized as a pair of algorithms $\langle \Psi_{\text{SVD}(|\mathcal{U}_2|,\varepsilon)}, \Phi_{\text{SVD}(|\mathcal{U}_2|,\varepsilon)}\rangle$:

$$Y'_u = \Psi_{\text{SVD}(|\mathcal{U}_2|,\varepsilon)}(Y_u) \triangleq \Psi_{\ell_2(\varepsilon)}(Y_u), \tag{22}$$

$$\Phi_{\text{SVD}(|\mathcal{U}_2|,\varepsilon)}\left(\{Y'_u\}_{u\in\mathcal{U}_2}\right) \triangleq X_3. \tag{23}$$

Here, $\Psi_{\ell_2(\varepsilon)}$ is the perturbation algorithm of the $\ell_2$-ball mechanism [13].

## 4.5 Phase 4: Final HHI

In this phase, the aggregator applies SVIM over the reduced candidate domain $X_3$ using users in $\mathcal{U}_3$, and identifies the top-$k$ heavy hitters.

This phase can be formalized as $\langle \Psi_{\text{HHI}(|\mathcal{U}_3|,\varepsilon)}, \Phi_{\text{HHI}(|\mathcal{U}_3|,\varepsilon)}\rangle$:

$$y_u = \Psi_{\text{HHI}(|\mathcal{U}_3|,\varepsilon)}(\mathbf{x}_u) \triangleq \Psi_{\text{SVIM}(\varepsilon)}(\mathbf{x}_u \cap X_3), \tag{24}$$

$$\Phi_{\text{HHI}(|\mathcal{U}_3|,\varepsilon)}\left(\{y_u\}_{u\in\mathcal{U}_3}\right) \triangleq \Phi_{\text{SVIM}}\left(\{y_u\}_{u\in\mathcal{U}_3}; \varepsilon, |X_3|\right). \tag{25}$$

The formal definition of our BSS-HHI method is given by Equations (6), (7), (13), (14), (22), (23), (24), and (25).

# 5 ANALYSIS

We establish the privacy, utility, and overhead properties of BSS-HHI. Detailed derivations are provided in the full version [1].

## 5.1 Privacy

Theorem 5.1. BSS-HHI *satisfies $\varepsilon$-LDP defined on user values.*

Proof. BSS-HHI partitions users into disjoint groups for different phases, and each user participates in exactly one phase and contributes at most one perturbed report. In Phases 1 and 2, users report perturbed BSS representations through SVIM [42]. In Phase 3, users report perturbed vectors through the $\ell_2$-Ball mechanism [13]. In Phase 4, users report perturbed values over the final candidate domain through SVIM. Both SVIM and the $\ell_2$-Ball mechanism satisfy $\varepsilon$-LDP. All subsequent candidate reconstruction, filtering, and ranking operations use only perturbed reports and therefore preserve privacy by post-processing. Since the user groups are disjoint, parallel composition gives $\varepsilon$-LDP for the complete protocol [14]. □

## 5.2 Utility

*5.2.1 Effect of Domain Encoding on Reconstruction Bias.* Let $V$ be the random value in the sampled data and $W_i = b_i(V)$ its $i$th encoded bit. By the chain rule, the true frequency of $v$ is

$$f(v) = \Pr(W_1 = b_1(v)) \prod_{i=2}^{l} \Pr(W_i = b_i(v) \mid W_{<i} = b_{<i}(v)), \tag{26}$$

whereas the BSS reconstruction score multiplies marginal bit probabilities,

$$\mathcal{P}(v) = \prod_{i=1}^{l} \Pr(W_i = b_i(v)). \tag{27}$$

Comparing the two scores and averaging their log difference under the true distribution gives

$$\mathcal{B}_0 = D_{\text{KL}}(f\|\mathcal{P}) = \sum_{i=2}^{l} I(W_i; W_{<i}). \tag{28}$$

Here, $\mathcal{B}_0$ is the Phase 1 reconstruction bias and is determined by the dependence among all encoded bits.

After Phase 2 conditions on a selected bit $W_1$, averaging the reconstruction bias within its conditional partitions gives

$$\mathcal{B}_1 = \sum_{i=2}^{l} I(W_i; W_{2:i-1} \mid W_1). \tag{29}$$

Here, $\mathcal{B}_1$ is the Phase 2 reconstruction bias and is determined by the dependence among the remaining bits given the selected bit. Different domain encodings change the dependence among all bits used in Phase 1 and the conditional dependence among the remaining bits used in Phase 2. The reconstruction bias in both phases therefore depends on the domain encoding.

*5.2.2 Imbalance-Based Bit Selection.* Imbalance is used to select expansion bits. For two patterns differing only at bit $j$, with $\hat{p}_{j,b} \in (0,1)$, that bit contributes

$$L_j = \left|\log \frac{\hat{p}_{j,1}}{\hat{p}_{j,0}}\right| = 2\,\text{arctanh}\left(2\left|\hat{p}_{j,1} - \frac{1}{2}\right|\right) \tag{30}$$

to their absolute log-score ratio. This contribution increases monotonically with imbalance. Selecting the most imbalanced bit removes the largest single-bit contribution to cross-partition score differences. Candidate scores are therefore not excessively increased or decreased by that bit.

*5.2.3 Candidate Recall Guarantee.* In Phase 1 and within each Phase 2 partition $\tau$, BSS-HHI reconstructs and ranks values over the corresponding domain and retains the top $m_\tau$ values. Let $t_\tau$ be the number of unfixed bits, $n_\tau$ the reports used for each bit estimate, $q_\tau = \Pr(V \in \mathcal{X}^{(\tau)})$ the partition mass, and $\alpha = \min_{h\in H_k} \Pr(V = h)$ the minimum sampled probability among the top-$k$ heavy hitters.

Define the encoding-dependent bias $\mathcal{B}_\tau = D_{\mathrm{KL}}(f_\tau \| \mathcal{P}_\tau)$ and the LDP score error

$$E_\tau = O\left(\frac{\log^2 d}{\varepsilon q_\tau}\sqrt{\frac{\log(k\log d/\delta_\tau)}{n_\tau}}\right). \tag{31}$$

Theorem 5.2 (Candidate recall). *Assuming $\alpha^{t_\tau} > 2E_\tau$, with probability at least $1-\delta_\tau$, every top-$k$ item in this partition is retained when*

$$m_\tau \geq \min\left\{|\mathcal{X}^{(\tau)}|, \left\lceil \frac{1}{\max\{\alpha^{t_\tau}, \alpha - \sqrt{\mathcal{B}_\tau/2}\} - 2E_\tau} \right\rceil\right\}. \tag{32}$$

*This bound reflects the given domain encoding through $\mathcal{B}_\tau$. Since $\mathcal{P}_\tau(h) \geq f_\tau(h)^{t_\tau} \geq \alpha^{t_\tau}$ for every encoding, an encoding-independent worst-case condition is*

$$m_\tau \geq \min\left\{|\mathcal{X}^{(\tau)}|, \left\lceil \frac{1}{\alpha^{t_\tau} - 2E_\tau} \right\rceil\right\}. \tag{33}$$

*If every heavy hitter belongs to at least one Phase 1 or Phase 2 domain satisfying the corresponding condition, then*

$$\Pr[H_k \subseteq X_1 \cup X_2] \geq 1 - \sum_\tau \delta_\tau. \tag{34}$$

*Proof sketch.* On the LDP concentration event, every estimated reconstruction score differs from its exact score by at most $E_\tau$. Thus, only a value with exact score at least $\mathcal{P}_\tau(h) - 2E_\tau$ can have an estimated score no smaller than that of a true heavy hitter $h$. The exact reconstruction scores sum to at most one, so fewer than $1/(\mathcal{P}_\tau(h) - 2E_\tau)$ values can meet this threshold; retaining this many values therefore retains $h$. For a given domain encoding, Pinsker's inequality gives $\mathcal{P}_\tau(h) \geq \alpha - \sqrt{\mathcal{B}_\tau/2}$, while $\mathcal{P}_\tau(h) \geq \alpha^{t_\tau}$ holds for every domain encoding. These two lower bounds yield Eqs. (32) and (33); a union bound over the corresponding retention events gives Eq. (34). The complete derivation is provided in [1].

## 5.3 Communication and Computation Costs

In Phase 1, each user in $\mathcal{U}_1$ samples one value, constructs its BSS representation, and sends one perturbed SVIM report, incurring $O(l)$ computation and $O(1)$ communication per user. The aggregator runs SVIM over the bit domain with $O(|\mathcal{U}_1| + l)$ computation and $O(|\mathcal{U}_1|)$ communication. In Phase 2, each user still sends a constant-size SVIM report for conditional bit estimation in the assigned partition. Thus, for a bounded number of expansion iterations, the per-user computation and communication costs remain $O(l)$ and $O(1)$, respectively. Across the conditional partitions $\mathcal{T}$, the aggregator requires $O(\sum_{\tau\in\mathcal{T}}(|\mathcal{U}_1^{(\tau)}| + l))$ computation and $O(\sum_{\tau\in\mathcal{T}}|\mathcal{U}_1^{(\tau)}|)$ communication. In Phase 3, each user in $\mathcal{U}_2$ constructs an indicator matrix over $X_1 \cup X_2$, computes $G_r^\top I_u H_r$, receives the low-rank factors, and sends a perturbed vector of length $r^2$. The resulting user computation is $O(|X_1 \cup X_2|r)$ and communication is $O(\sqrt{|X_1 \cup X_2|}r + r^2)$. Across $T$ rounds, the aggregator requires $O(T(|X_1 \cup X_2|^{3/2} + |X_1 \cup X_2|r) + |\mathcal{U}_2|r^2)$ computation and $O(|\mathcal{U}_2|\sqrt{|X_1 \cup X_2|}r + |\mathcal{U}_2|r^2)$ communication. In Phase 4, each user in $\mathcal{U}_3$ sends one SVIM report over $X_3$, incurring $O(1)$ computation and communication, while the aggregator requires $O(|\mathcal{U}_3| + |X_3|)$ computation and $O(|\mathcal{U}_3|)$ communication.

# 6 EVALUATION

In this section, we describe the experimental setup, including evaluation metrics, datasets, baselines, and parameter settings, followed by the experimental results of BSS-HHI.

## 6.1 Experiment Setup

*6.1.1 Environment.* All experiments are conducted on a desktop with an Intel Ultra 7 265K 3.90 GHz CPU and 64GB memory. Each experiment is performed 10 times.

*6.1.2 Metrics.* To comprehensively evaluate the performance of BSS-HHI and baselines, we adopt three metrics:

**(1) F1 Score [31]** evaluates whether the correct values are identified. Let $\widehat{S}_k$ denote the set of top-$k$ identified values, and $S_k^\star$ denote the set of true top-$k$ values. Pre $= |\widehat{S}_k \cap S_k^\star|/|\widehat{S}_k|$, Rec $= |\widehat{S}_k \cap S_k^\star|/|S_k^\star|$, and F1 $= 2 \cdot$ Pre $\cdot$ Rec$/$(Pre + Rec).

**(2) NCR [43]** further accounts for the ground-truth ranks of the identified values:

$$\mathrm{NCR} = \sum_{v\in\widehat{S}_k} q(v) \,/\, \big(k(k+1)/2\big), \tag{35}$$

where $q(v) = k - r + 1$ if $v$ appears in $S_k^\star$ with rank $r$, and $q(v) = 0$ otherwise.

**(3) MCR** quantifies the covered frequency mass:

$$\mathrm{MCR} = \sum_{x\in\widehat{S}_k\cap S_k^\star} f(x) \,/\, \sum_{x\in S_k^\star} f(x). \tag{36}$$

*6.1.3 Datasets.* We evaluate BSS-HHI and baselines on seven datasets, randomly sampling 100,000 users per experimental run, except for FedUB [53].

**(1) Kosarak** [20] contains 990,002 users and **41,270** distinct website IDs; **(2) MovieLens** [22] contains 162,414 users and **209,163** distinct movie IDs; **(3) UserBeh**avior [53] contains 863,327 users and **528,150** distinct product IDs; **(4) Zipf-65k** contains **65,536** values with frequencies following a Zipf distribution ($\alpha = 1.5$), where 70% of users have set sizes uniformly sampled from 1 to 5 and the remaining 30% follow a Zipf distribution ($\alpha = 1.5$); **(5) Zipf-131k** follows the same generation process as Zipf-65k and contains **131,072** values; **(6) IFTTT** [11, 24] contains **354** distinct values; **(7) FedUB** [53] is constructed from **UserBehavior**, containing 600,000 records partitioned into three parties with 435,637, 154,213, and 10,150 users, sharing **5,089** common values.

*6.1.4 Baselines.* We include state-of-the-art (SOTA) LDP methods covering HHI, relation mining, and federated HHI as baselines. **(1) SVIM [42]** is a frequent value (item) mining protocol that is described in Section 2.3. **(2) SVD-FO [11]** is a frequency estimation protocol that maps the value domain into a matrix and applies LDP-RM to estimate value frequencies, subsequently selecting the top-$k$ heavy hitters. **(3) PemSet [56]** is an HHI protocol under LDP that extends PEM [43] to set-valued data. PemSet supports four LDP protocols for multiset frequency estimation (termed as MOLH, MOLH-S, MPCKV, and MWheel). For a fair comparison, we use MWheel, which achieves the best performance. **(4) ARBSF [12]** is a set-valued HHI protocol under LDP that formulates HHI as a multi-armed bandit problem. **(5) LDP-RM [11]** is a SOTA relation

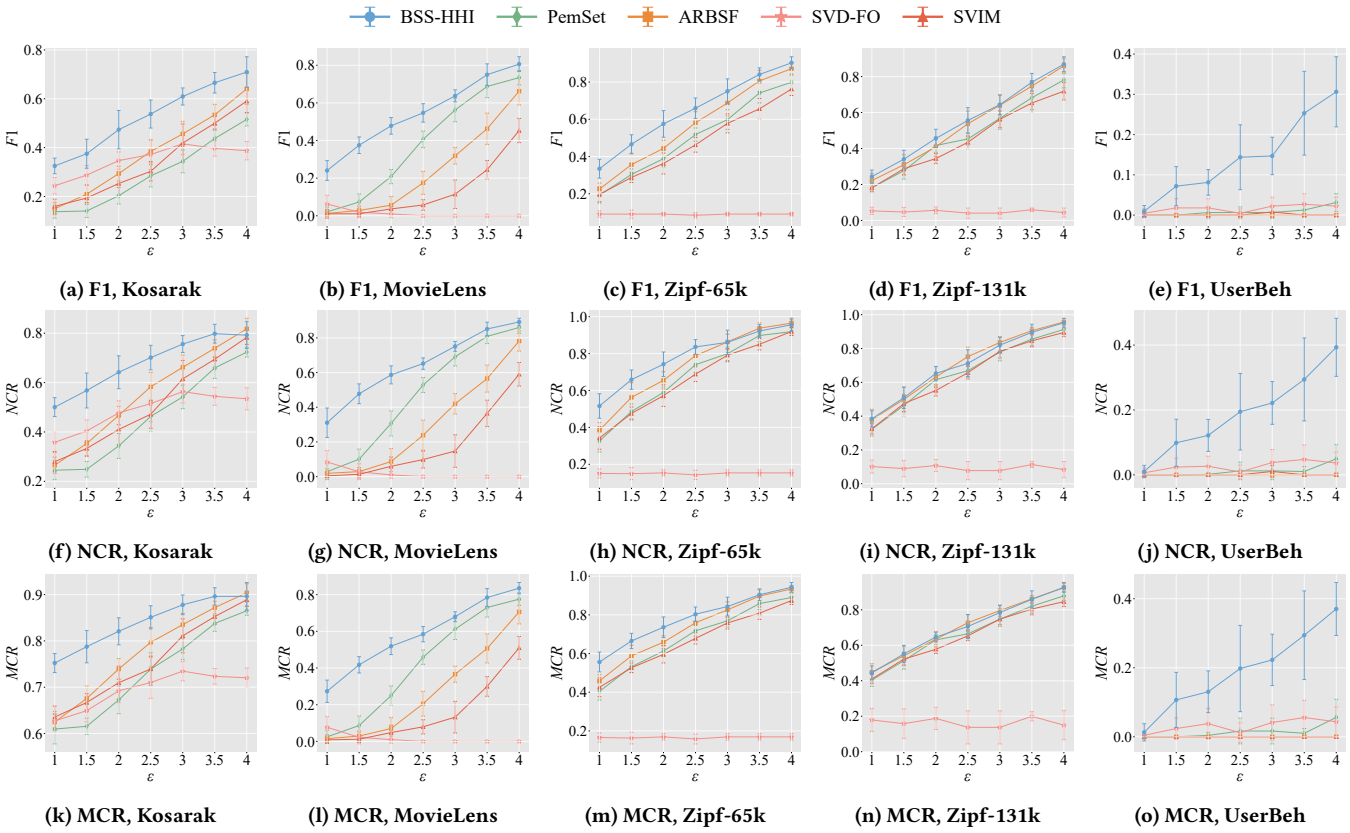



**Figure 3: Results of E1: Performance (F1, NCR, and MCR) of BSS-HHI and baselines on different $\varepsilon$ with $k = 32$.**

mining method under LDP. **(6) TAPS [51]** is a SOTA federated HHI method under LDP.

*6.1.5 Parameter settings.* For all baselines, we use their recommended parameter settings. For PemSet, we follow its original preprocessing by padding or truncating each user's value set to size 10. The parameters of BSS-HHI follow the guideline in **[E4]**; dataset-specific configurations are available in the repository [1].

## 6.2 Experiments and Results

We evaluate BSS-HHI through the utility-privacy trade-off, parameter sensitivity, and two case studies on relation mining and federated HHI. More detailed parameter and ablation experiments are reported in [1].

*6.2.1 Utility-privacy trade-off.* We evaluate the utility-privacy trade-off of BSS-HHI against baselines through experiments **[E1]** to **[E2]**.

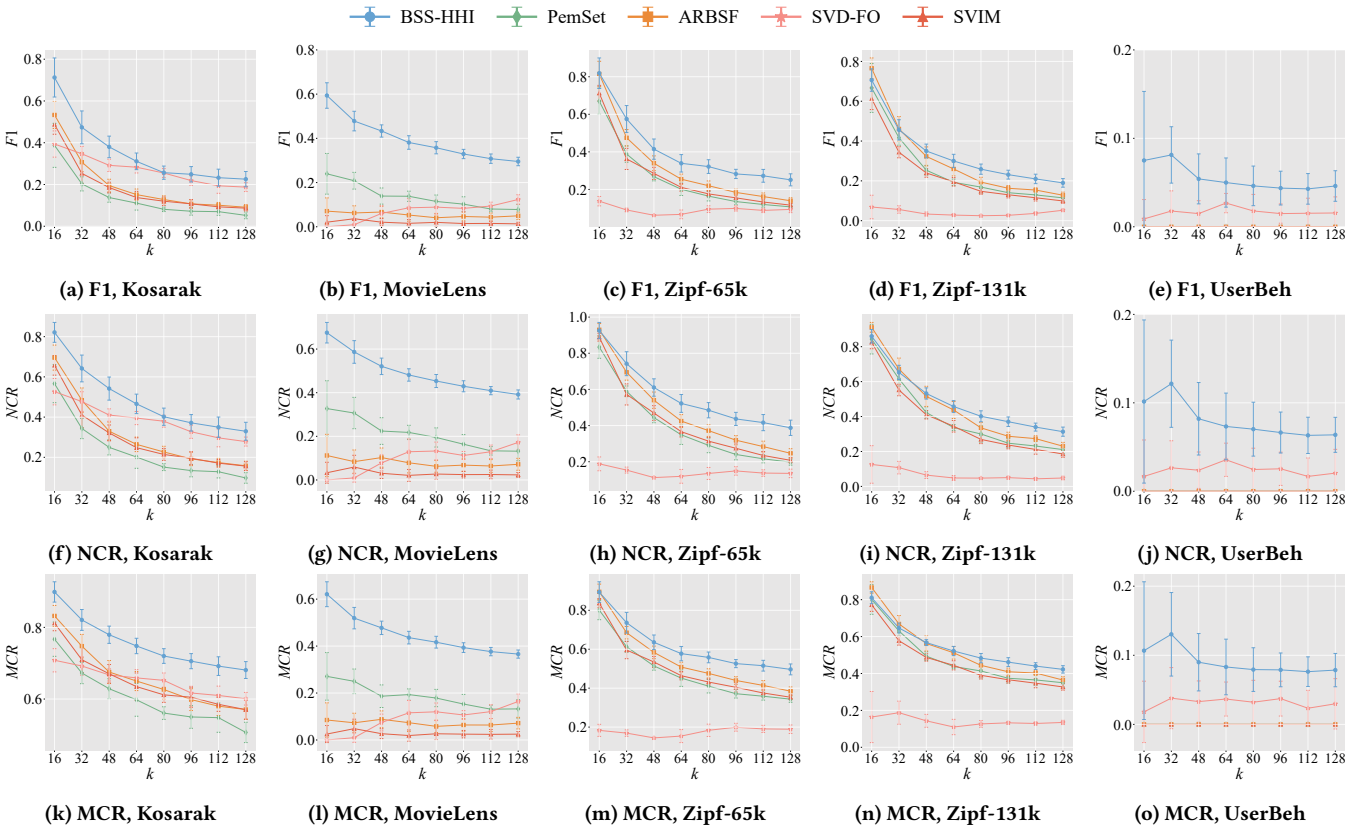
**[E1] Performance under different $\varepsilon$.** In this experiment, we compare BSS-HHI with baselines on three real-world and two synthetic datasets under different privacy budgets. We fix $k = 32$ and set $\varepsilon \in \{1, 1.5, \ldots, 4\}$. The results are shown in Figure 3. We observe that BSS-HHI consistently outperforms the baselines. The advantage is particularly pronounced on UserBeh, where the extremely large domain and small frequency gaps make existing methods nearly ineffective, while BSS-HHI remains effective.

**[E2] Performance under different $k$.** In this experiment, we compare BSS-HHI with baselines while varying the top-$k$ parameter. We fix $\varepsilon = 2$ and set $k \in \{16, 32, \ldots, 128\}$. The results are shown in Figure 4. We observe that as $k$ increases, F1, NCR, and MCR decrease, as less frequent values become harder to identify accurately. Despite this trend, BSS-HHI consistently achieves the best overall performance.

We also conduct candidate recall experiments across phases in **[E7]** and an SVD filter ablation in **[E9]**. The results in Table 2 and Figure 11 of [1] illustrate the trade-off: although Phase 3 reduces candidate recall, it provides a smaller domain for final HHI, ultimately improving identification accuracy.

*6.2.2 Effect of parameters.* We conduct experiments to evaluate the effects of user-group allocation ratios, candidate domain sizes, and the selected bit sequences for Phase 2.

**[E3] Effect of user-group allocation ratios.** In this experiment, we study the effect of different user-group allocation ratios on the performance of BSS-HHI. We evaluate 13 representative ratios under a fixed $\varepsilon = 3$ and $k \in \{16, 32\}$. The results are shown in Figure 5. Each ratio is specified as $a : b : c$ indicating the proportions of users assigned to the three user-groups, respectively. The partition ratio $1:1:8$ consistently achieves the best performance across all datasets and evaluation metrics, and we therefore set it as the default ratio.




(a) F1, Kosarak (b) F1, MovieLens (c) F1, Zipf-65k (d) F1, Zipf-131k (e) F1, UserBeh

(f) NCR, Kosarak (g) NCR, MovieLens (h) NCR, Zipf-65k (i) NCR, Zipf-131k (j) NCR, UserBeh

(k) MCR, Kosarak (l) MCR, MovieLens (m) MCR, Zipf-65k (n) MCR, Zipf-131k (o) MCR, UserBeh

**Figure 4: Results of E2: Performance (F1, NCR, and MCR) of `BSS-HHI` and baselines on different $k$ with $\varepsilon = 2$.**

**Table 1: Results of E4: Parameter selection guideline.**

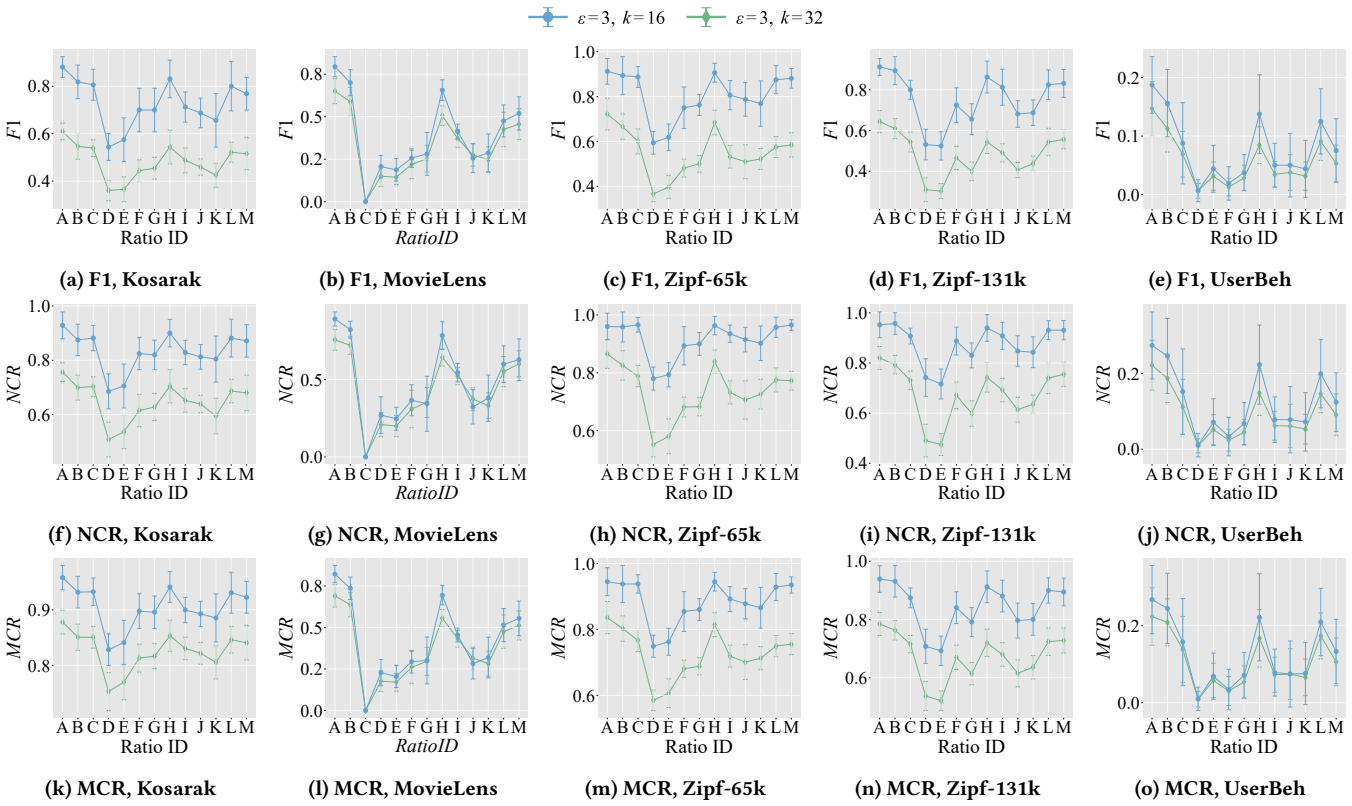
| Parameter | Selection rule |
|---|---|
| $\mathbf{c}$ | Select 2-6 bits in the first conditional expansion and use a non-increasing number thereafter. At lower $\varepsilon$, use Phase 2 to reduce the candidate domain. As $\varepsilon$ increases, select fewer bits or omit Phase 2 to retain more candidates. Recommended schedules include $(2, 1, 1)$, $(6, 2)$, and $(6, 5, 3, 1)$. |
| $\lvert X_1\rvert/d \times 100\%$ | Use 0.5%-5% when followed by Phase 2 and 10%-20% when using Phase 1 only. At smaller $\varepsilon$, choose a value near 0.5% with Phase 2 or near 10% with Phase 1 only; as $\varepsilon$ increases, increase it toward 5% or 20%, respectively. |
| $C_0$ | Use $C_0 = 50$ by default. |
| $\rho_1$ | Set $\rho_1 = \lvert X_1 \cup X_2\rvert/d$ after candidate reconstruction. |
| $\rho_2$ | Choose from $\{0.1, 0.3, 0.5, 0.7\}$ so that $\rho_1\rho_2$ retains approximately 1%-4% of the domain. Use a smaller retained fraction at lower $\varepsilon$ and increase it as $\varepsilon$ permits more reliable final identification. |
| $\theta$ | Use $\theta = 0.9$ by default. |
| $T$ | Use $T = 5$ by default. |

**[E4] Effect of parameters.** In this experiment, we study the effect of parameters, including the conditional expansion bit sequence $\mathbf{c}$, the Phase 1 candidate-domain ratio $|X_1|/d \times 100\%$, the Phase 2 candidate size parameter $C_0$, the retention ratio after Phases 1 and 2 $\rho_1 = |X_1 \cup X_2|/d \times 100\%$, the Phase 3 retention ratio $\rho_2 = |X_3|/|X_1 \cup X_2| \times 100\%$, the low-rank threshold $\theta$, and the number of SVD iterations $T$. We use $k \in \{16, 32\}$ and $\varepsilon \in \{2, 4\}$.

We use a sequence $\mathbf{c} = (c^{(1)}, \ldots, c^{(q)})$ to specify the number of bits selected in each iteration of conditional expansion, where $c^{(t)}$ denotes the number of bits selected in the $t$-th iteration. Each selected bit independently partitions every current partition into two sub-partitions. Therefore, each partition produced after the $t$-th iteration contains $t$ specified bits. For example, with $l = 4$, $\mathbf{c} = (2, 1)$ first selects two bits, e.g., $b_1$ and $b_2$. Bit $b_1$ produces $1 * **$ and $0 * **$, while $b_2$ produces $*1 * *$ and $*0 * *$. The second iteration selects one additional bit, e.g., $b_4$, producing two new partitions from each current partition, e.g., $1 * *1$ and $1 * *0$.

The parameter $C_0$ denotes the number of candidates reconstructed in each partition during the first conditional expansion iteration. For later iterations, the number of candidates reconstructed in each partition is reduced to $0.7C_0$, $0.5C_0$, $0.3C_0$, and $0.2C_0$, respectively.

Figure 6 presents the conditional-expansion schedules across all five datasets and privacy settings (Zipf-131K performs best using Phase 1 only). The candidate-capacity and retention results, together with the SVD-parameter results, are reported in [1]. Table 1 summarizes the resulting selection guideline.

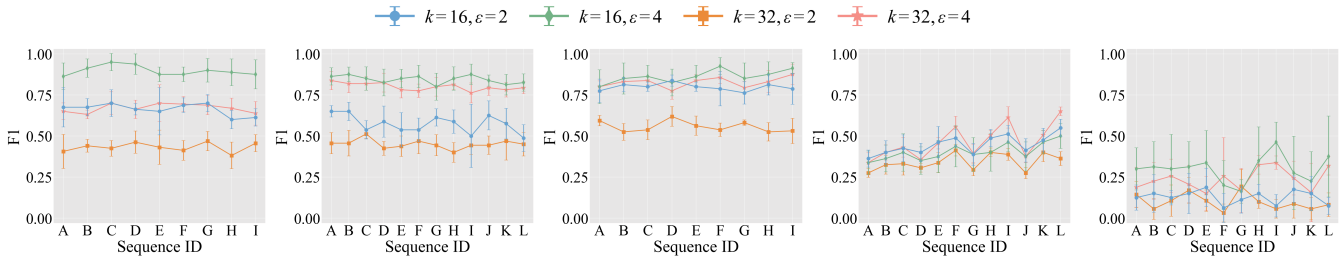



(a) F1, Kosarak (b) F1, MovieLens (c) F1, Zipf-65k (d) F1, Zipf-131k (e) F1, UserBeh

(f) NCR, Kosarak (g) NCR, MovieLens (h) NCR, Zipf-65k (i) NCR, Zipf-131k (j) NCR, UserBeh

(k) MCR, Kosarak (l) MCR, MovieLens (m) MCR, Zipf-65k (n) MCR, Zipf-131k (o) MCR, UserBeh

**Figure 5: Results of E3: Performance (F1, NCR, and MCR) of BSS-HHI with different user group allocation ratios. Ratio IDs correspond to the following three-group user partition ratios: A** $(1:1:8)$**, B** $(2:2:6)$**, C** $(3:3:4)$**, D** $(8:1:1)$**, E** $(1:8:1)$**, F** $(6:2:2)$**, G** $(2:6:2)$**, H** $(2:3:5)$**, I** $(5:2:3)$**, J** $(3:5:2)$**, K** $(4:4:2)$**, L** $(2:4:4)$**, and M** $(4:2:4)$**.**



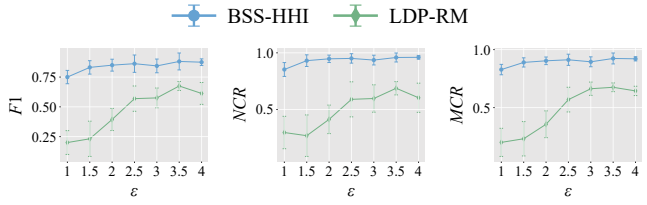

(a) Kosarak (b) MovieLens (c) Zipf-65K (d) Zipf-131K (e) UserBehavior

**Figure 6: Results of E4: Effect of the conditional-expansion sequence c on F1. Results use** $k \in \{16, 32\}$ **and** $\varepsilon \in \{2, 4\}$**. Sequence IDs denote A** $(2)$**, B** $(4)$**, C** $(6)$**, D** $(2,1)$**, E** $(4,2)$**, F** $(6,2)$**, G** $(2,1,1)$**, H** $(4,2,1)$**, I** $(6,3,1)$**, J** $(2,1,1,1)$**, K** $(4,3,2,1)$**, and L** $(6,5,3,1)$**.**

*6.2.3 Case Studies of BSS-Based Reconstruction.* We conduct two case studies to evaluate the performance of BSS-based candidate reconstruction in relation mining and federated HHI.

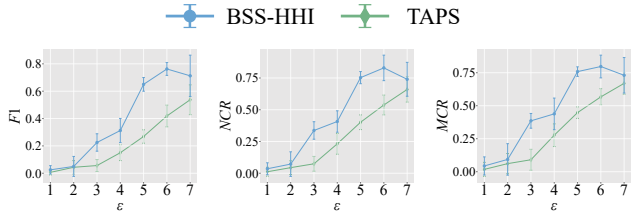
**[E5] Performance on relation mining.** In this experiment, we extend BSS-HHI to the relation mining task and compare it with the state-of-the-art LDP relation mining method LDP-RM [11] on IFTTT. We set $\varepsilon \in \{1, 1.5, \ldots, 4\}$ and $k = 16$, and results are shown in Figure 7. Since relation mining operates on binary relations between pairs of values, we first map each relation to a one-dimensional value by treating all $354^2$ possible relations as values, and then apply BSS-HHI on the transformed domain. We observe that BSS-HHI consistently outperforms LDP-RM across all evaluated privacy budgets in this case study.

**[E6] Performance on federated HHI.** In this experiment, we extend BSS-HHI to the federated HHI task and compare it with the state-of-the-art federated HHI method TAPS [51]. We consider $\varepsilon \in \{1, 2, \ldots, 7\}$ with $k = 16$, and the results are shown in Figure 8. In our implementation, each party independently runs BSS-HHI to




(a) F1, IFTTT (b) NCR, IFTTT (c) MCR, IFTTT

**Figure 7: Results of E5: Performance (F1, NCR, and MCR) of BSS-HHI and LDP-RM on relation mining task under different $\varepsilon$ with $k = 16$.**




(a) F1, FedUB (b) NCR, FedUB (c) MCR, FedUB

**Figure 8: Results of E6: Performance (F1, NCR, and MCR) of BSS-HHI and TAPS on federated HHI task under different $\varepsilon$ with $k = 16$.**

identify its local top-$10k$ candidates, and the server then aggregates the candidates from all parties to derive the global top-$k$ heavy hitters. We observe that BSS-HHI consistently outperforms TAPS across all evaluated privacy budgets in this case study.

## 7 RELATED WORK

We survey frequency estimation, HHI, and data mining under LDP.

**Frequency Estimation under LDP.** Frequency estimation is a fundamental task in LDP data mining [17, 23, 44, 48]. RAPPOR combines Bloom filters and randomized response to estimate user values and has been deployed in Chrome [6, 16]. Bassily et al. [5] construct succinct histograms, while Kairouz et al. [28] study k-RR and O-RR for discrete distribution estimation. Wang et al. [41] provide a unified framework for pure LDP mechanisms and derive optimized OUE and OLH protocols. Other work improves estimates through calibration and prior information [27], categorical mechanisms [39], and set-valued publication [40]. Xue et al. [46] use difference trees to address change points in continual data collection under LDP. For large domains, Apple's Count Mean Sketch hashes each value into a compact sketch, while its Hadamard variant reduces each privatized report to one bit [2]. PGR uses projective geometry to encode reports and reconstruct the frequency vector efficiently [19]. MSS combines residue-number encoding with Subset Selection to reduce communication and decoding costs, while Pan derives optimized sketch constructions for large domains [3, 32].

**HHI under LDP.** Heavy hitter identification (HHI) identifies the most frequent values under LDP [37]. HHI protocols typically construct a candidate domain to avoid estimating and ranking over the entire domain. LDPMiner uses separate candidate selection and refinement stages [33]. Existing domain-pruning strategies include padding-and-sampling and reconstruction from partial encodings [4, 18, 42, 43]. PEM extends frequent prefixes, while PemSet adapts this process to set-valued data [43, 56]. Prefix-tree methods and BSS-HHI both use public value encodings to construct candidates. Prefix trees are effective when heavy hitters are concentrated under a few short prefixes, whereas BSS-HHI can further use common patterns at later or nonconsecutive bit positions and across prefix branches without retaining complete preceding prefix paths. TreeHist uses hierarchical prefix search, whereas Bitstogram combines hashing with bitwise reconstruction [4]. Bun et al. [7] and Wu and Wirth [45] develop single-valued HHI protocols with recovery guarantees through expander- and parameterized-sketch constructions. These methods assume one value per user, unlike the set-valued setting considered by PemSet and BSS-HHI. Federated HHI has been studied through sampling, trie-based aggregation, and federated analytics [9, 15, 51, 54], while HG-LDP addresses HHI over data streams [30]. We adapt BSS-HHI to single-valued data and compare it with single-valued HHI methods in [1].

**Data Mining under LDP.** Numerous studies have investigated various data mining tasks under LDP. These include frequent itemset mining [38, 42, 50], which aims to discover commonly co-occurring items; marginal release [10, 34, 52], which focuses on releasing low-dimensional marginals of high-dimensional data; key-value collection [21, 47], which gathers key-value pairs under LDP; and relation mining [11], which explores relations between items under LDP. Other works study more complex tasks on more complex data, such as graph data analysis under LDP [25, 26, 35, 55].

## 8 CONCLUSION

In this paper, we study HHI under LDP in large domains, where unreliable ranking is a key challenge behind the accuracy degradation of FO-based methods. To address this challenge, we propose the BSS-strategy, which reconstructs candidates from bit-level statistics without directly ranking noisy frequency estimates over the entire domain, and design the multi-phase BSS-HHI protocol to further refine the candidate domain and identify heavy hitters accurately under LDP. Both theoretical analysis and extensive experiments demonstrate that BSS-HHI significantly improves identification accuracy in large-domain settings. We also evaluate BSS-based reconstruction through case studies on relation mining and federated HHI. Future work will study its applicability to other analytics and learning problems under LDP.

## ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China Grant Nos. 62472089, 62232004, and 92467205, US National Science Foundation (NSF) Awards 2325451, Jiangsu Provincial Key Laboratory of Network and Information Security Grant No. BM2003201, Key Laboratory of Computer Network and Information Integration of Ministry of Education of China Grant No. 93K-9, and Collaborative Innovation Center of Novel Software Technology and Industrialization. Any opinions, findings, conclusions, and recommendations in this paper are those of the authors and do not necessarily reflect the views of the funding agencies.

# REFERENCES


[1] 2026. Full Version, with Source Code Availability. https://github.com/zybxav/BSS-HHI.

[2] Apple Differential Privacy Team. 2017. Learning with Privacy at Scale. Apple Machine Learning Research.

[3] Héber Hwang Arcolezi. 2026. Private Frequency Estimation via Residue Number Systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[4] Raef Bassily, Kobbi Nissim, Uri Stemmer, and Abhradeep Thakurta. 2017. Practical locally private heavy hitters. In *Annual Conference on Neural Information Processing Systems (NeurIPS 2017)*. 2288–2296.

[5] Raef Bassily and Adam Smith. 2015. Local, private, efficient protocols for succinct histograms. In *Proceedings of the forty-seventh annual ACM symposium on Theory of computing (STOC 2015)*. 127–135.

[6] Burton H Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* 13, 7 (1970), 422–426.

[7] Mark Bun, Jelani Nelson, and Uri Stemmer. 2019. Heavy Hitters and the Structure of Local Privacy. *ACM Transactions on Algorithms* 15, 4 (2019), 1–40.

[8] Karan Chadha, Junye Chen, John Duchi, Vitaly Feldman, Hanieh Hashemi, Omid Javidbakht, Audra McMillan, and Kunal Talwar. 2024. Differentially Private Heavy Hitter Detection using Federated Analytics. In *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. 512–533. https://doi.org/10.1109/SaTML59370.2024.00032

[9] Graham Cormode and Akash Bharadwaj. 2022. Sample-and-threshold differential privacy: Histograms and applications. In *International Conference on Artificial Intelligence and Statistics (AISTATS 2022)*. PMLR, 1420–1431.

[10] Graham Cormode, Tejas Kulkarni, and Divesh Srivastava. 2018. Marginal release under local differential privacy. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD 2018)*. 131–146.

[11] Kai Dong, Zheng Zhang, Chuang Jia, Zhen Ling, Ming Yang, Junzhou Luo, and Xinwen Fu. 2024. Relation mining under local differential privacy. In *33rd USENIX Security Symposium (USENIX Security 2024)*. 955–972.

[12] Rong Du, Qingqing Ye, Yue Fu, Haibo Hu, and Kai Huang. 2024. Top-k discovery under local differential privacy: An adaptive sampling approach. *IEEE Transactions on Dependable and Secure Computing* (2024).

[13] John C Duchi, Michael I Jordan, and Martin J Wainwright. 2018. Minimax optimal procedures for locally private estimation. *J. Amer. Statist. Assoc.* 113, 521 (2018), 182–201.

[14] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science* 9, 3–4 (2014), 211–407.

[15] Ahmed Roushdy Elkordy, Yahya H Ezzeldin, Shanshan Han, Shantanu Sharma, Chaoyang He, Sharad Mehrotra, Salman Avestimehr, et al. 2023. Federated analytics: A survey. *APSIPA Transactions on Signal and Information Processing* 12, 1 (2023).

[16] Úlfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. 2014. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *ACM SIGSAC Conference on Computer and Communications Security (CCS 2014)*. 1054–1067.

[17] Huiyu Fang, Liquan Chen, Yali Liu, and Yuan Gao. 2023. Locally differentially private frequency estimation based on convolution framework. In *2023 IEEE Symposium on Security and Privacy (S&P 2023)*. IEEE, 2208–2222.

[18] Giulia Fanti, Vasyl Pihur, and Úlfar Erlingsson. 2015. Building a RAPPOR with the unknown: Privacy-preserving learning of associations and data dictionaries. *arXiv preprint arXiv:1503.01214* (2015).

[19] Vitaly Feldman et al. 2022. Private Frequency Estimation via Projective Geometry. In *Proceedings of the International Conference on Machine Learning*.

[20] Bart Goethals. 2003. Frequent itemset mining dataset repository. *http://fimi.cs.helsinki.fi/data/* (2003).

[21] Xiaolan Gu, Ming Li, Yueqiang Cheng, Li Xiong, and Yang Cao. 2020. {PCKV}: Locally differentially private correlated {Key-Value} data collection with optimized utility. In *29th USENIX security symposium (USENIX security 2020)*. 967–984.

[22] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4, Article 19 (Dec. 2015), 19 pages. https://doi.org/10.1145/2827872

[23] Fang Huiyu, Chen Liquan, and Liu Suhui. [n.d.]. Further Study on Frequency Estimation under Local Differential Privacy. In *34th USENIX security symposium (USENIX Security 2025)*.

[24] IFTTT. Accessed Apr. 2023. Top Applets of 2023. [online], https://ifttt.com/explore/top-applets-2023.

[25] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. {Communication-Efficient} triangle counting under local differential privacy. In *31st USENIX security symposium (USENIX Security 22)*. 537–554.

[26] Huanhuan Jia, Yuanbo Zhao, Kai Dong, Zhen Ling, Ming Yang, Junzhou Luo, and Xinwen Fu. [n.d.]. Distributed Private Aggregation in Graph Neural Networks. In *34th USENIX security symposium (USENIX Security 2025)*.

[27] Jinyuan Jia and Neil Zhenqiang Gong. 2019. Calibrate: Frequency estimation and heavy hitter identification with local differential privacy via incorporating prior knowledge. In *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. IEEE, 2008–2016.

[28] Peter Kairouz, Keith Bonawitz, and Daniel Ramage. 2016. Discrete distribution estimation under local privacy. In *International Conference on Machine Learning (ICML 2016)*. PMLR, 2436–2444.

[29] Shiva Prasad Kasiviswanathan, Homin K Lee, Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2011. What can we learn privately? *SIAM Journal on Computing (SICOMP)* 40, 3 (2011), 793–826.

[30] Xiaochen Li, Weiran Liu, Jian Lou, Yuan Hong, Lei Zhang, Zhan Qin, and Kui Ren. 2024. Local differentially private heavy hitter detection in data streams with bounded memory. *Proceedings of the ACM on Management of Data (SIGMOD 2024)* 2, 1 (2024), 1–27.

[31] Christopher D Manning. 2008. *Introduction to information retrieval*. Syngress Publishing,.

[32] Mingen Pan. 2026. Strict Optimality of Frequency and Distribution Estimation Under Local Differential Privacy. *arXiv preprint arXiv:2603.11523* (2026).

[33] Zhan Qin, Yin Yang, Ting Yu, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2016. Heavy hitter estimation over set-valued data with local differential privacy. In *ACM SIGSAC Conference on Computer and Communications Security (CCS 2016)*. 192–203.

[34] Xuebin Ren, Chia-Mu Yu, Weiren Yu, Shusen Yang, Xinyu Yang, Julie A. McCann, and Philip S. Yu. 2018. LoPub : High-Dimensional Crowdsourced Data Publication With Local Differential Privacy. *IEEE Transactions on Information Forensics and Security (TIFS)* 13, 9 (2018), 2151–2166.

[35] Sina Sajadmanesh and Daniel Gatica-Perez. 2021. Locally private graph neural networks. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security (CCS 2021)*. 2130–2145.

[36] Charles Steinfield, Thomas Adelaar, and Fang Liu. 2005. Click and mortar strategies viewed from the web: a content analysis of features illustrating integration between retailers' online and offline presence. *Electronic Markets* 15, 3 (2005), 199–212.

[37] Ning Wang, Xiaokui Xiao, Yin Yang, Ta Duy Hoang, Hyejin Shin, Junbum Shin, and Ge Yu. 2018. PrivTrie: Effective frequent term discovery under local differential privacy. In *2018 IEEE 34th International Conference on Data Engineering (ICDE 2018)*. IEEE, 821–832.

[38] Shaowei Wang, Liusheng Huang, Yiwen Nie, Pengzhan Wang, Hongli Xu, and Wei Yang. 2018. Privset: Set-valued data analyses with local differential privacy. In *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 1088–1096.

[39] Shaowei Wang, Liusheng Huang, Yiwen Nie, Xinyuan Zhang, Pengzhan Wang, Hongli Xu, and Wei Yang. 2019. Local differential private data aggregation for discrete distribution estimation. *IEEE Transactions on Parallel and Distributed Systems (TPDS)* 30, 9 (2019), 2046–2059.

[40] Shaowei Wang, Yuqiu Qian, Jiachun Du, Wei Yang, Liusheng Huang, and Hongli Xu. 2020. Set-valued data publication with local privacy: Tight error bounds and efficient mechanisms. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1234–1247.

[41] Tianhao Wang, Jeremiah Blocki, Ninghui Li, and Somesh Jha. 2017. Locally differentially private protocols for frequency estimation. In *26th USENIX security symposium (USENIX Security 2017)*. 729–745.

[42] Tianhao Wang, Ninghui Li, and Somesh Jha. 2018. Locally differentially private frequent itemset mining. In *IEEE Symposium on Security and Privacy (S&P 2018)*. 127–143.

[43] Tianhao Wang, Ninghui Li, and Somesh Jha. 2021. Locally differentially private heavy hitter identification. *IEEE Transactions on Dependable and Secure Computing (TDSC)* 18, 2 (2021), 982–993.

[44] Stanley L Warner. 1965. Randomized response: A survey technique for eliminating evasive answer bias. *Journal of the American statistical association* 60, 309 (1965), 63–69.

[45] Hao Wu and Anthony Wirth. 2022. Asymptotically Optimal Locally Private Heavy Hitters via Parameterized Sketches. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*.

[46] Qiao Xue, Qingqing Ye, Haibo Hu, Youwen Zhu, and Jian Wang. 2022. DDRM: A continual frequency estimation mechanism with local differential privacy. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 35, 7 (2022), 6784–6797.

[47] Qingqing Ye, Haibo Hu, Xiaofeng Meng, and Huadi Zheng. 2019. PrivKV: Key-value data collection with local differential privacy. In *2019 IEEE Symposium on Security and Privacy (S&P 2019)*. IEEE, 317–331.

[48] Yutong Ye, Tianhao Wang, Min Zhang, and Dengguo Feng. 2025. Revisiting EM-based Estimation for Locally Differentially Private Protocols. In *32nd Annual Network and Distributed System Security Symposium (NDSS 2025)*. 24–28.

[49] Xi Zhang, Jian Cheng, Ting Yuan, Biao Niu, and Hanqing Lu. 2013. TopRec: domain-specific recommendation through community topic mining in social network. In *Proceedings of the 22nd international conference on World Wide Web (WWW 2013)*. 1501–1510.

[50] Xinyuan Zhang, Liusheng Huang, Peng Fang, Shaowei Wang, Zhenyu Zhu, and Hongli Xu. 2017. Differentially private frequent itemset mining from smart devices in local setting. In *International Conference on Wireless Algorithms, Systems,*

*and Applications (WASA 2017)*. Springer, 433–444.

[51] Yuemin Zhang, Qingqing Ye, and Haibo Hu. 2025. Federated Heavy Hitter Analytics with Local Differential Privacy. *Proceedings of the ACM on Management of Data (SIGMOD 2025)* 3, 1 (2025), 1–27.

[52] Zhikun Zhang, Tianhao Wang, Ninghui Li, Shibo He, and Jiming Chen. 2018. Calm: Consistent adaptive local marginal for marginal release under local differential privacy. In *ACM SIGSAC Conference on Computer and Communications Security (CCS 2018)*. 212–229.

[53] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1079–1088.

[54] Wennan Zhu, Peter Kairouz, Brendan McMahan, Haicheng Sun, and Wei Li. 2020. Federated heavy hitters discovery with differential privacy. In *International Conference on Artificial Intelligence and Statistics (AISTATS 2020)*. PMLR, 3837–3847.

[55] Xiaochen Zhu, Vincent YF Tan, and Xiaokui Xiao. 2023. Blink: link local differential privacy in graph neural networks via Bayesian estimation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS 2023)*. 2651–2664.

[56] Youwen Zhu, Yiran Cao, Qiao Xue, Qihui Wu, and Yushu Zhang. 2023. Heavy hitter identification over large-domain set-valued data with local differential privacy. *IEEE Transactions on Information Forensics and Security (TIFS)* 19 (2023), 414–426.

# A DETAILED ANALYSIS OF RECONSTRUCTION BIAS

We provide the complete derivation underlying the reconstruction-bias results summarized in Section 5.2.1. Let $V$ be the random value over $\mathcal{X}$, and let $W_i = b_i(V)$ denote the random variable corresponding to its $i$th encoded bit. For a value $v \in \mathcal{X}$, write $b(v) = (b_1(v), \ldots, b_l(v))$. By the chain rule,

$$f(v) = \Pr(V = v) = \Pr(W_1 = b_1(v)) \prod_{i=2}^{l} \Pr(W_i = b_i(v) \mid W_{<i} = b_{<i}(v)), \tag{37}$$

where $W_{<i} = (W_1, \ldots, W_{i-1})$. In contrast, the BSS strategy reconstructs the score using marginal bit probabilities:

$$\mathcal{P}(v) = \prod_{i=1}^{l} \Pr(W_i = b_i(v)). \tag{38}$$

The pointwise log discrepancy is therefore

$$\begin{aligned} \Delta_0(v) &= \log \frac{f(v)}{\mathcal{P}(v)} \\ &= \sum_{i=2}^{l} \log \frac{\Pr(W_i = b_i(v) \mid W_{<i} = b_{<i}(v))}{\Pr(W_i = b_i(v))}. \end{aligned} \tag{39}$$

Taking the expectation under the true distribution gives the aggregate bias

$$\begin{aligned} \mathcal{B}_0 &= D_{\mathrm{KL}}(f \| \mathcal{P}) = \mathbb{E}_{V \sim f}[\Delta_0(V)] \\ &= \sum_{i=2}^{l} I(W_i; W_{<i}). \end{aligned} \tag{40}$$

Thus, the discrepancy introduced by the product score is the accumulated dependence between each bit and its preceding bits. If the encoded bits are mutually independent, then $\mathcal{B}_0 = 0$. Otherwise, replacing the conditional probabilities in Eq. (37) with the marginals in Eq. (38) produces a nonzero reconstruction bias. Because the domain encoding changes the joint distribution of the encoded bits, it can also change $\mathcal{B}_0$.

We next consider a single-bit conditional expansion on $W_1$. Within each partition specified by $W_1 = b_1(v)$, the reconstruction score is

$$\mathcal{P}^{(\tau)}(v) = \prod_{i=2}^{l} \Pr(W_i = b_i(v) \mid W_1 = b_1(v)). \tag{41}$$

For a KL-based comparison over the complete domain, define the corresponding global score

$$\widetilde{\mathcal{P}}_1(v) = \Pr(W_1 = b_1(v)) \mathcal{P}^{(\tau)}(v). \tag{42}$$

Within either partition, the first factor is constant and therefore does not affect the candidate ranking. The residual reconstruction bias is

$$\mathcal{B}_1 = D_{\mathrm{KL}}(f \| \widetilde{\mathcal{P}}_1) = \sum_{i=2}^{l} I(W_i; W_{2:i-1} \mid W_1). \tag{43}$$

Thus, $\mathcal{B}_0$ is the Phase 1 reconstruction bias determined by dependence among all encoded bits, whereas $\mathcal{B}_1$ is the Phase 2 reconstruction bias determined by the conditional dependence among the remaining bits given $W_1$. Different domain encodings change these two forms of dependence, which determine the discrepancy introduced by the corresponding product reconstruction. The reconstruction bias in each phase therefore depends on the domain encoding. These encoding-dependent biases are incorporated into the candidate-recall bound in Appendix B.

# B PROOF OF THE CANDIDATE RECALL GUARANTEE

We analyze the recall guarantee by bounding how many candidates can have estimated reconstruction scores no smaller than that of each true top-$k$ heavy hitter. A sufficient retained set size follows from this count, which depends on the reconstruction bias between the true frequency and reconstruction score together with the LDP estimation error in that score.

## B.1 Candidate Recall through Score Comparison

Let $\tau$ denote the fixed-bit conditions used in candidate reconstruction: $\tau = \varnothing$ represents Phase 1, and each Phase 2 partition gives another $\tau$. BSS-HHI reconstructs and ranks values over the corresponding domain $\mathcal{X}^{(\tau)}$. Let $f_\tau(v)$ be the sampled conditional frequency, $S_\tau^\pi(v)$ the reconstruction score under domain encoding $\pi$, and $\widehat{S}_\tau^\pi(v)$ its LDP estimate. If the top $m_\tau$ values are retained, then for a heavy hitter $h \in \mathcal{X}^{(\tau)}$ define

$$C_\tau(h) = \left| \{ v \in \mathcal{X}^{(\tau)} : \widehat{S}_\tau^\pi(v) \geq \widehat{S}_\tau^\pi(h) \} \right|.$$

The candidate reconstruction for $\tau$ retains $h$ whenever

$$m_\tau \geq C_\tau(h). \tag{44}$$

It remains to bound $C_\tau(h)$. Consider the event, shown below to hold with probability at least $1 - \delta_\tau$, on which the LDP estimation error is at most $E_\tau$ for every value. Any $v$ counted by $C_\tau(h)$ then satisfies

$$S_\tau^\pi(v) \geq \widehat{S}_\tau^\pi(v) - E_\tau \geq \widehat{S}_\tau^\pi(h) - E_\tau \geq S_\tau^\pi(h) - 2E_\tau. \tag{45}$$

Thus, LDP noise can change the pairwise score difference by at most $2E_\tau$. The bias analysis below gives

$$S_\tau^\pi(h) \geq \max\left\{ f_\tau(h)^{t_\tau}, f_\tau(h) - \beta_\tau^\pi \right\},$$

where $t_\tau$ is the number of unfixed encoded-bit probabilities multiplied in partition $\tau$ and $\beta_\tau^\pi$ bounds the downward reconstruction bias. Write $p_{j,b} = \Pr(W_j = b \mid W \in \tau)$ for an unfixed bit $j$. Over all assignments of the unfixed bits, the exact product scores satisfy

$$\sum_{b \in \{0,1\}^{t_\tau}} S_\tau^\pi(b) = \prod_{j=1}^{t_\tau} (p_{j,0} + p_{j,1}) = 1.$$

The legal IDs in $\mathcal{X}^{(\tau)}$ form a subset of these assignments, so their scores sum to at most one. Hence there can be at most the reciprocal of the lower bound in Eq. (45) such values. Therefore, when this lower bound is positive,

$$\Pr\left[ C_\tau(h) \leq \min\left\{ |\mathcal{X}^{(\tau)}|, \frac{1}{\max\{ f_\tau(h)^{t_\tau}, f_\tau(h) - \beta_\tau^\pi \} - 2E_\tau} \right\} \right] \geq 1 - \delta_\tau. \tag{46}$$

If the lower bound is nonpositive, reconstruction bias and LDP noise may together erase the score guaranteed for $h$, so this argument gives only the trivial bound $C_\tau(h) \leq |\mathcal{X}^{(\tau)}|$. We next bound the reconstruction bias and LDP noise in Eq. (46).

## B.2 Bounding Reconstruction Bias

Let $H_k$ be the true top-$k$ set and let $\alpha = \min_{h \in H_k} \Pr(V = h)$ be the minimum sampled probability among the items whose recall is required. Section 5.2.1 and Appendix A analyze the aggregate reconstruction bias

$$\mathcal{B}_\tau^\pi = D_{\mathrm{KL}}\left(f_\tau \middle\| S_\tau^\pi\right), \qquad \beta_\tau^\pi := \sup_{v \in \mathcal{X}^{(\tau)}} [f_\tau(v) - S_\tau^\pi(v)]_+ \le \sqrt{\frac{\mathcal{B}_\tau^\pi}{2}}, \tag{47}$$

where $[x]_+ = \max\{x, 0\}$ and the inequality follows from Pinsker's inequality. This immediately gives the encoding-dependent bound used in the recall analysis:

$$S_\tau^\pi(h) \ge \alpha - \sqrt{\mathcal{B}_\tau^\pi/2}, \qquad h \in H_k \cap \mathcal{X}^{(\tau)}. \tag{48}$$

Different domain encodings change the joint and conditional distributions of the encoded bits and hence $\mathcal{B}_\tau^\pi$, which appears in the encoding-dependent retention condition below.

We further provide a bound that holds for any domain encoding. Within partition $\tau$, every bit event matching $h$ includes the event $V = h$, so every corresponding conditional bit probability is at least $f_\tau(h)$. Therefore,

$$S_\tau^\pi(h) \ge f_\tau(h)^{t_\tau} \ge \alpha^{t_\tau}, \qquad h \in H_k \cap \mathcal{X}^{(\tau)}, \tag{49}$$

or, equivalently, the downward bias satisfies

$$[f_\tau(h) - S_\tau^\pi(h)]_+ \le f_\tau(h) - f_\tau(h)^{t_\tau}. \tag{50}$$

Here $f_\tau(h) \ge \Pr(V = h) \ge \alpha$ because conditioning divides the global frequency by the probability of the partition containing $h$. Equation (48) is tighter when the encoded distribution has a small reconstruction bias, whereas Eq. (49) remains valid under arbitrary remapping. Together they yield the maximum appearing in Eq. (46).

## B.3 Bounding LDP Noise

Let $n_\tau$ be the number of reports used for each conditional bit estimate associated with $\tau$, and $q_\tau = \Pr(V \in \mathcal{X}^{(\tau)}) > 0$ its probability mass. If all joint frequencies and the partition mass are estimated within $\eta_{\mathrm{SVIM}} \le q_\tau/2$, the clipped conditional estimates have errors at most $4\eta_{\mathrm{SVIM}}/q_\tau$, so the product score changes by at most $4t_\tau\eta_{\mathrm{SVIM}}/q_\tau$ because all probabilities lie in $[0, 1]$. Applying the SVIM concentration bound to the joint frequencies and partition mass gives

$$E_\tau = O\left(\frac{\log^2 d}{\varepsilon q_\tau}\sqrt{\frac{\log(k \log d/\delta_\tau)}{n_\tau}}\right), \tag{51}$$

with probability at least $1 - \delta_\tau$, where $t_\tau \le \lceil \log_2 d \rceil$. Hence $E_\tau$ decreases as the number of participating users or $\varepsilon$ increases, and increases with the encoded bit width.

## B.4 Retained Set Size and Recall Guarantee

Let $\mathcal{T}$ index Phase 1 and the Phase 2 partitions used for candidate reconstruction. Under $\alpha^{t_\tau} > 2E_\tau$, Equations (44) and (46) show that a heavy hitter is retained for $\tau$ if $m_\tau$ satisfies

$$m_\tau \ge \min\left\{|\mathcal{X}^{(\tau)}|, \left\lceil \frac{1}{\max\left\{\alpha^{t_\tau}, \alpha - \sqrt{\mathcal{B}_\tau^\pi/2}\right\} - 2E_\tau} \right\rceil\right\}, \tag{52}$$

Equation (52) is the encoding-dependent bound for the given domain encoding. It uses $\mathcal{B}_\tau^\pi$ and can therefore give a tighter sufficient value of $m_\tau$.

Using Eq. (49) alone removes the encoding-dependent bias term and gives the alternative sufficient condition

$$m_\tau \ge \min\left\{|\mathcal{X}^{(\tau)}|, \left\lceil \frac{1}{\alpha^{t_\tau} - 2E_\tau} \right\rceil\right\}, \tag{53}$$

Equation (53) is the encoding-independent bound for arbitrary domain encodings. It is a worst-case condition and can require retaining more candidates.

The two equations are alternative sufficient lower bounds on the same $m_\tau$. If every $h \in H_k$ belongs to at least one Phase 1 or Phase 2 domain satisfying either bound, then all top-$k$ heavy hitters are retained whenever the corresponding LDP concentration events hold. A union bound over these events gives

$$\Pr[H_k \subseteq X_1 \cup X_2] \ge 1 - \sum_{\tau \in \mathcal{T}} \delta_\tau. \tag{54}$$

Thus, the two $m_\tau$ inequalities are sufficient size conditions, and Eq. (54) is the resulting joint recall guarantee.

For Phase 1, $t_\tau = \lceil \log_2 d \rceil$ and $n_\tau$ is its preassigned share of the total $n$ users. The total number retained before deduplication is $\sum_{\tau \in \mathcal{T}} m_\tau$. Duplicates only reduce $|X_1 \cup X_2|$.

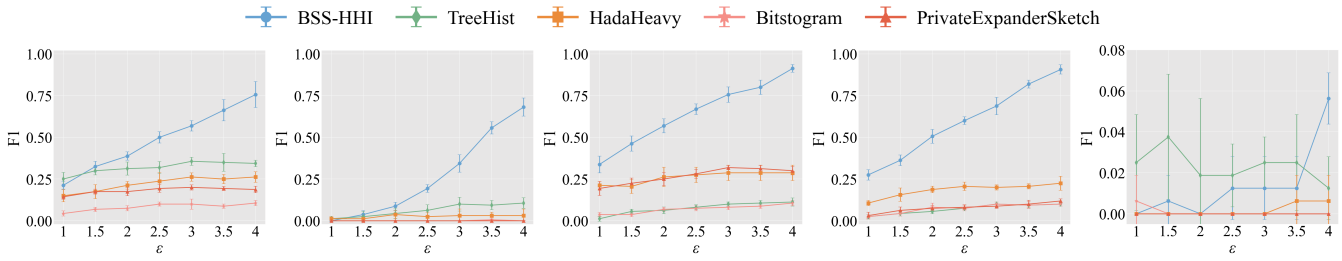
# C ADDITIONAL EXPERIMENTS

**Experimental settings.** Unless stated otherwise, the supplementary experiments use the five datasets, implementations, privacy model, and evaluation metrics defined in Section 6.1. We use the same main-experiment parameter configurations and vary only the component or parameter under study. Points and table entries report the mean, and error bars or ± values report the standard deviation. The captions specify $k$ and $\varepsilon$ when they differ across experiments.

**[E7] Candidate recall by phase.** We measure whether the true top-$k$ values remain in the candidate domain after each successive phase. Phases 1 and 2 retain high recall on four datasets, while conditional expansion improves recall on the more challenging UserBehavior data. Phase 3 further reduces the candidate domain, balancing candidate recall against the accuracy of final identification.

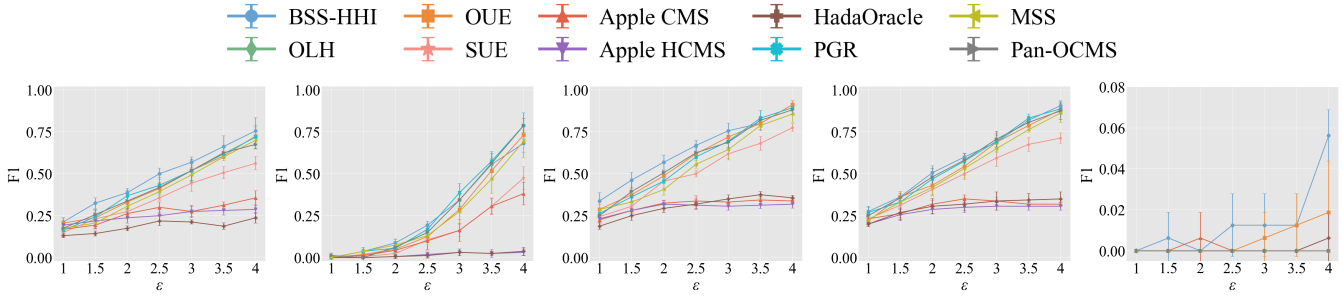
**[E8] Single-valued HHI and FO comparison.** For methods with public implementations, we retain their core mechanisms and adapt only the input and output interfaces. For methods without public implementations, we implement the mechanisms described in their papers. We uniformly sample one item from every nonempty user record using a fixed seed and recompute the true top-$k$ on the resulting single-valued data. All methods use the same converted data, number of users, privacy budget, and trial seeds. For BSS-HHI, we retain candidate reconstruction and replace the final SVIM estimator with OLH.

Compared with the evaluated end-to-end HHI methods in Figure 9, BSS-HHI provides the best overall performance. Compared with frequency-oracle methods in Figure 10, BSS-HHI remains competitive overall and performs better on several large-domain datasets.




(a) Kosarak (b) MovieLens (c) Zipf-65K (d) Zipf-131K (e) UserBehavior

**Figure 9: Results of E8: Performance (F1) of BSS-HHI and end-to-end HHI methods with $k = 32$ under varying $\varepsilon$.**




(a) Kosarak (b) MovieLens (c) Zipf-65K (d) Zipf-131K (e) UserBehavior

**Figure 10: Results of E8: Performance (F1) of BSS-HHI and frequency-oracle methods with $k = 32$ under varying $\varepsilon$.**

**Table 2: Results of E7: Candidate recall after successive phases with $k = 32$ and $\varepsilon \in \{1, 2, 3, 4\}$.**

| Dataset | $\varepsilon$ | Phase 1 Recall | Phase 1/2 Recall | Phase 3 Recall |
|---|---|---|---|---|
| Kosarak | 1 | 0.875 ± 0.105 | 0.887 ± 0.078 | 0.741 ± 0.096 |
| | 2 | 0.947 ± 0.034 | 0.953 ± 0.032 | 0.775 ± 0.091 |
| | 3 | 0.972 ± 0.026 | 0.972 ± 0.026 | 0.809 ± 0.033 |
| | 4 | 0.984 ± 0.016 | 0.984 ± 0.016 | 0.791 ± 0.052 |
| MovieLens | 1 | 0.759 ± 0.046 | 0.763 ± 0.045 | 0.697 ± 0.040 |
| | 2 | 0.822 ± 0.034 | 0.822 ± 0.034 | 0.766 ± 0.047 |
| | 3 | 0.969 ± 0.000 | 0.969 ± 0.000 | 0.859 ± 0.021 |
| | 4 | 0.969 ± 0.000 | 0.969 ± 0.000 | 0.831 ± 0.045 |
| Zipf-65K | 1 | 0.925 ± 0.064 | 0.938 ± 0.056 | 0.909 ± 0.083 |
| | 2 | 0.959 ± 0.034 | 0.966 ± 0.029 | 0.966 ± 0.029 |
| | 3 | 0.978 ± 0.028 | 0.984 ± 0.025 | 0.975 ± 0.034 |
| | 4 | 0.981 ± 0.032 | 0.981 ± 0.032 | 0.978 ± 0.040 |
| Zipf-131K | 1 | 1.000 ± 0.000 | 1.000 ± 0.000 | 0.775 ± 0.230 |
| | 2 | 1.000 ± 0.000 | 1.000 ± 0.000 | 0.944 ± 0.076 |
| | 3 | 1.000 ± 0.000 | 1.000 ± 0.000 | 0.994 ± 0.013 |
| | 4 | 1.000 ± 0.000 | 1.000 ± 0.000 | 0.991 ± 0.014 |
| UserBehavior | 1 | 0.497 ± 0.208 | 0.694 ± 0.135 | 0.575 ± 0.171 |
| | 2 | 0.666 ± 0.114 | 0.794 ± 0.066 | 0.503 ± 0.292 |
| | 3 | 0.650 ± 0.074 | 0.753 ± 0.063 | 0.584 ± 0.271 |
| | 4 | 0.531 ± 0.172 | 0.725 ± 0.090 | 0.656 ± 0.176 |

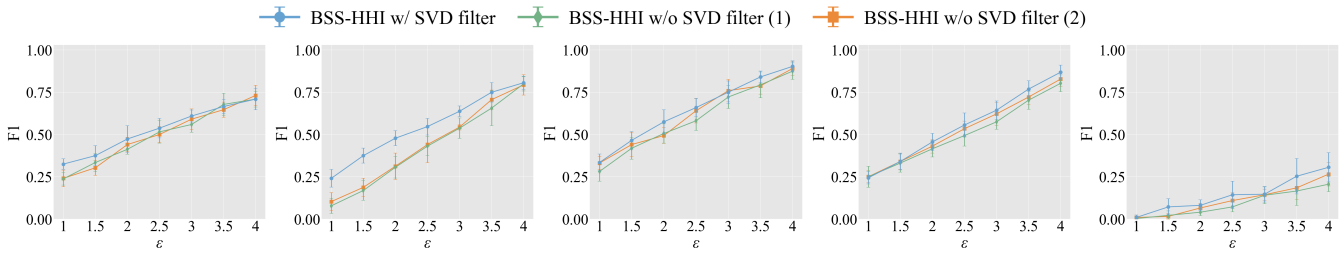
**[E9] Ablation of the SVD filter.** We compare BSS-HHI with two variants that remove the SVD filter and reassign its users to candidate reconstruction or final identification. Retaining the filter gives better overall performance.

## C.1 Parameter Sensitivity

We evaluate the parameters below and derive the selection rules in E4 from these results. **[E10]** varies the Phase 1 and conditional-expansion candidate capacities. Performance is generally stable around the default capacities, while the results show the expected trade-off between retaining more candidates and reducing the domain for final identification. **[E11]** varies the Phase 3 retention ratio. Intermediate ratios provide a stable balance between candidate retention and final estimation. **[E12]** varies the low-rank threshold and **[E13]** varies the number of SVD iterations. Performance remains stable around the defaults, supporting $\theta = 0.9$ and $T = 5$. **[E14]** varies the number of participating users. Increasing the number of users reduces the effect of LDP noise for all methods, while BSS-HHI retains its advantage. **[E15]** varies the Zipf exponent. BSS-HHI remains stable as the distribution becomes flatter.

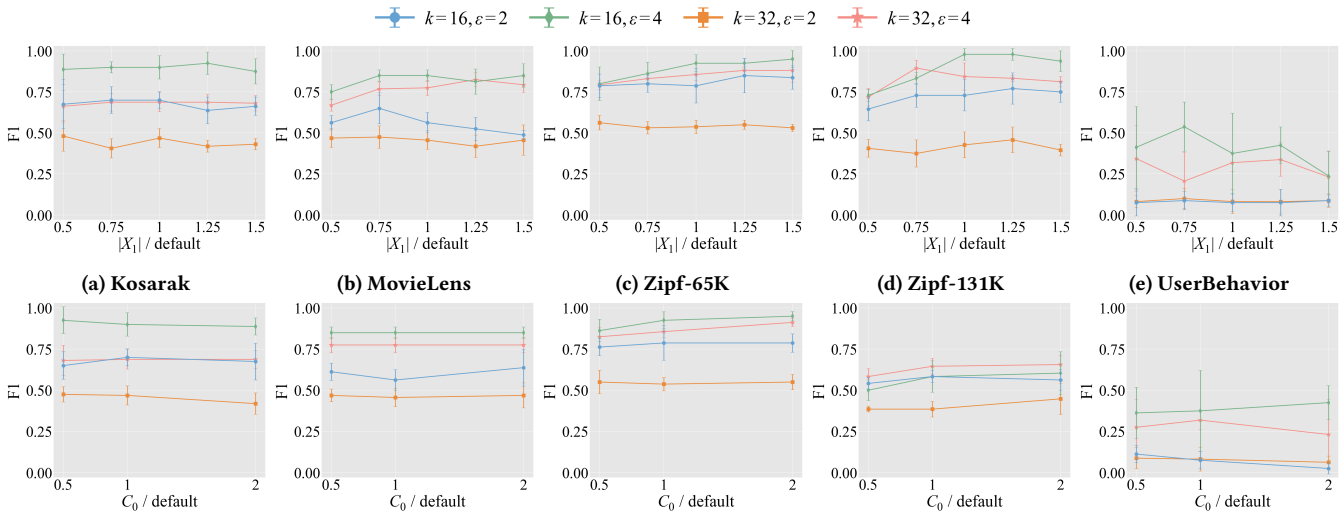
## C.2 Effect of Domain Encoding

**[E16] Original and fully random ID encodings.** We compare all methods under the original encoding and a fully random one-to-one remapping of the IDs. Table 3 shows that SVIM and ARBSF, which do not use the ID encoding to derive candidates, change little after remapping on the datasets with complete results. BSS-HHI and PEMSET both derive candidates from encoded IDs and therefore




(a) Kosarak (b) MovieLens (c) Zipf-65K (d) Zipf-131K (e) UserBehavior

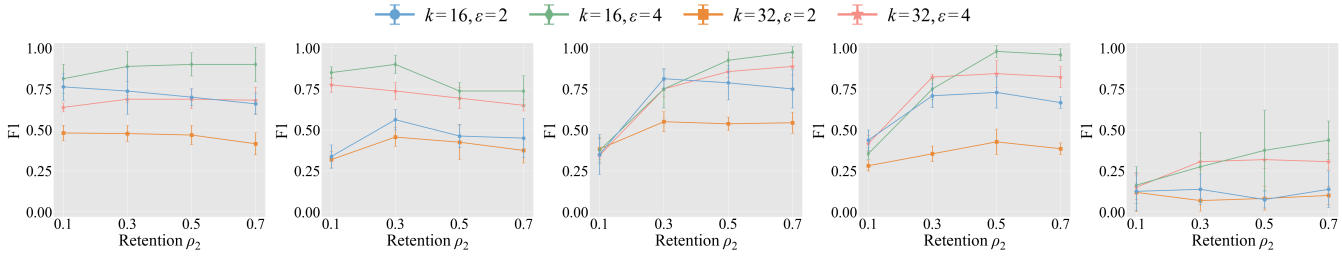
**Figure 11: Results of E9: Performance (F1) with and without the SVD filter for $k = 32$ under varying $\varepsilon$. The two variants reallocate the Phase 3 users to Phases 1/2 and Phase 4, respectively.**




(a) Kosarak (b) MovieLens (c) Zipf-65K (d) Zipf-131K (e) UserBehavior

(f) Kosarak (g) MovieLens (h) Zipf-65K (i) Zipf-131K (j) UserBehavior

**Figure 12: Results of E10: Effect of candidate capacities on F1. The top row varies $|X_1|$ over 0.5-1.5 times its default, and the bottom row varies $C_0$ over 0.5-2 times its default. Results use $k \in \{16, 32\}$ and $\varepsilon \in \{2, 4\}$.**

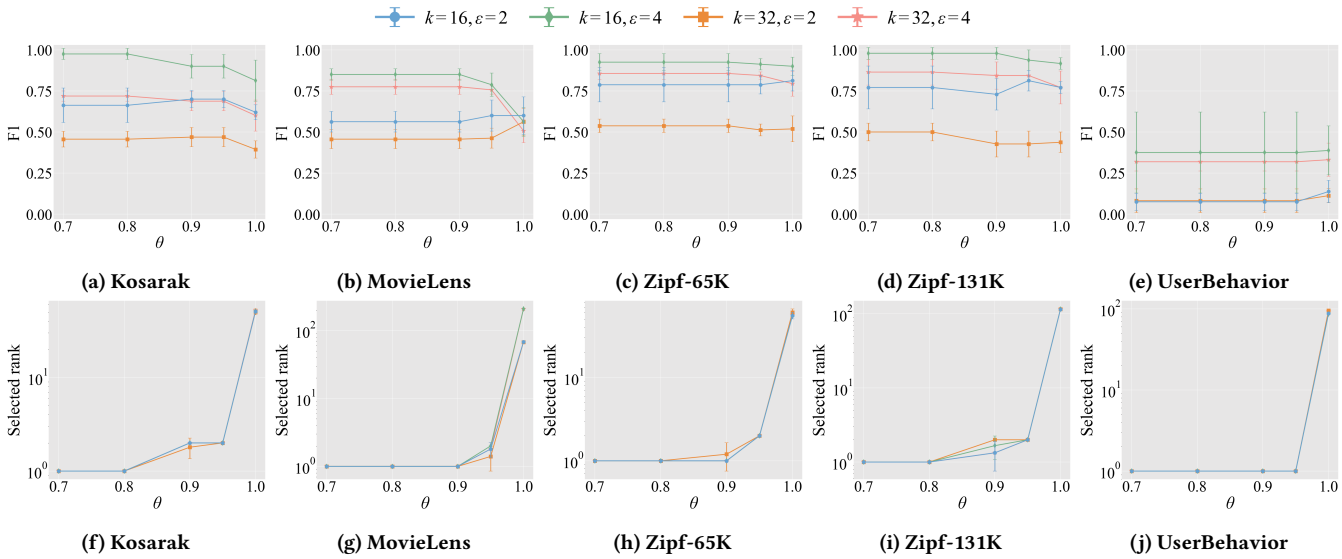



(a) Kosarak (b) MovieLens (c) Zipf-65K (d) Zipf-131K (e) UserBehavior

**Figure 13: Results of E11: Effect of the Phase 3 retention ratio $\rho_2$ on F1 with $k \in \{16, 32\}$ and $\varepsilon \in \{2, 4\}$.**

lose utility when the encoding is fully randomized. SVD-FO also declines because random remapping disperses the heavy hitters across its two-dimensional coordinates, making the high-frequency coordinate candidates in its first stage more difficult to identify.

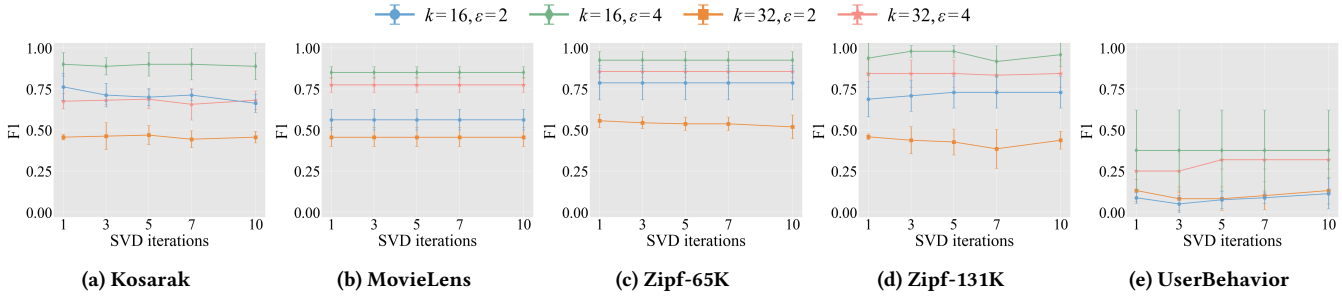
**[E17] Gradual ID permutation.** We gradually increase the ID permutation ratio from 0 to 1 for BSS-HHI and PEMSET. Figure 18 shows that both methods decline as more of the encoding is disrupted, while BSS-HHI degrades more slowly than PEMSET.




**(a) Kosarak** **(b) MovieLens** **(c) Zipf-65K** **(d) Zipf-131K** **(e) UserBehavior**

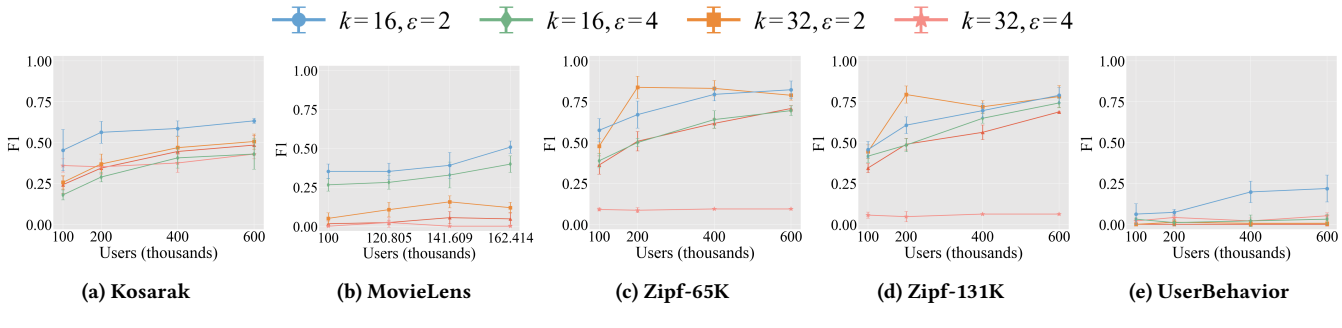
**(f) Kosarak** **(g) MovieLens** **(h) Zipf-65K** **(i) Zipf-131K** **(j) UserBehavior**

**Figure 14: Results of E12: Effect of the low-rank threshold $\theta$. The top row shows F1 and the bottom row shows selected rank on a logarithmic scale. Results use $k \in \{16, 32\}$ and $\varepsilon \in \{2, 4\}$.**




**(a) Kosarak** **(b) MovieLens** **(c) Zipf-65K** **(d) Zipf-131K** **(e) UserBehavior**

**Figure 15: Results of E13: Effect of the number of Phase 3 SVD iterations $T$ on F1 with $k \in \{16, 32\}$ and $\varepsilon \in \{2, 4\}$.**

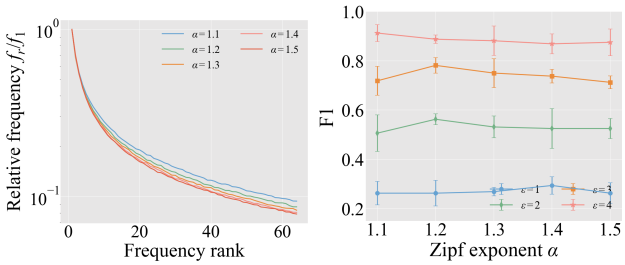



**(a) Kosarak** **(b) MovieLens** **(c) Zipf-65K** **(d) Zipf-131K** **(e) UserBehavior**

**Figure 16: Results of E14: Effect of the number of participating users on F1 for five methods with $\varepsilon = 2$ and $k = 32$.**

## C.3 Low-Rank Structure under Score Ordering

**[E18] Singular-value concentration and ordering sensitivity.** We compare descending-score ordering with random score permutations using retained energy, Rank@90%, and effective rank. Retained energy is $E_r = \sum_{i=1}^{r} \sigma_i^2 / \sum_i \sigma_i^2$. Rank@90% is the smallest $r$ with $E_r \geq 0.9$, and effective rank is $\exp(-\sum_i p_i \log p_i)$, where $p_i = \sigma_i / \sum_j \sigma_j$. Under descending-score ordering, the first singular component retains at least 96% of the energy and Rank@90% is one in every evaluated setting, confirming that these matrices are close to low rank. Random ordering substantially increases both



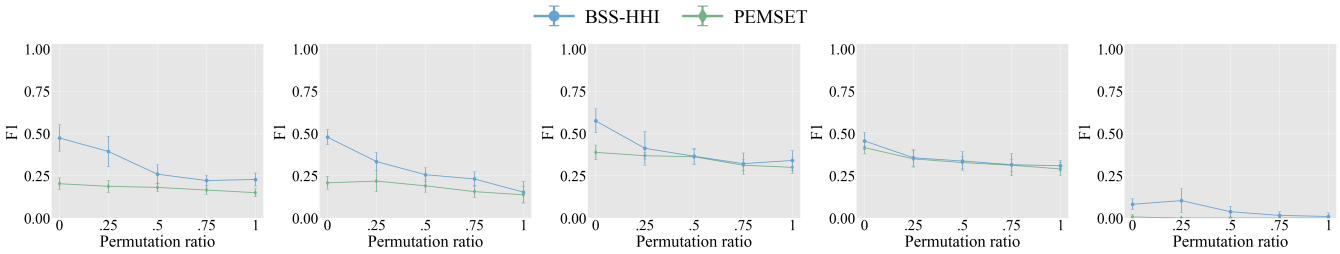

**(a) Head-frequency profile** **(b) Top-32 F1**

**Figure 17: Results of E15: Effect of the Zipf exponent $\alpha$ with $d = 2^{16}$, $k = 32$, and $\varepsilon \in \{1, 2, 3, 4\}$.**

Rank@90% and effective rank, showing that the specified ordering strengthens the low-rank concentration.

**Table 3: Results of E16: Performance (F1) under original and fully random ID encodings with $\varepsilon = 2$ and $k = 32$.**

| | BSS-HHI | | PEMSET | | SVIM | | ARBSF | | SVD-FO | |
|---|---|---|---|---|---|---|---|---|---|---|
| Dataset | Original | Random | Original | Random | Original | Random | Original | Random | Original | Random |
| Kosarak | 0.4736 ± 0.0786 | 0.2281 ± 0.0391 | 0.2031 ± 0.0338 | 0.1500 ± 0.0247 | 0.2531 ± 0.0473 | 0.2625 ± 0.0474 | 0.3009 ± 0.0342 | 0.3070 ± 0.0458 | 0.3469 ± 0.0355 | 0.0573 ± 0.0236 |
| MovieLens | 0.4781 ± 0.0443 | 0.1531 ± 0.0633 | 0.2083 ± 0.0378 | 0.1375 ± 0.0493 | 0.0364 ± 0.0281 | 0.0125 ± 0.0161 | 0.0656 ± 0.0473 | 0.0750 ± 0.0474 | 0.0094 ± 0.0200 | 0.0052 ± 0.0127 |
| Zipf-65K | 0.5750 ± 0.0715 | 0.3406 ± 0.0579 | 0.3884 ± 0.0437 | 0.3000 ± 0.0367 | 0.3625 ± 0.0562 | 0.3875 ± 0.0171 | 0.4774 ± 0.0452 | 0.4500 ± 0.0474 | 0.0907 ± 0.0094 | 0.0260 ± 0.0127 |
| Zipf-131K | 0.4563 ± 0.0508 | 0.3094 ± 0.0311 | 0.4167 ± 0.0378 | 0.2906 ± 0.0391 | 0.3438 ± 0.0279 | 0.3438 ± 0.0221 | 0.4440 ± 0.0422 | 0.4250 ± 0.0357 | 0.0562 ± 0.0188 | 0.0312 ± 0.0198 |
| UserBehavior | 0.0813 ± 0.0319 | 0.0094 ± 0.0211 | 0.0063 ± 0.0140 | 0.0000 ± 0.0000 | 0.0078 ± 0.0135 | 0.0000 ± 0.0000 | 0.0000 ± 0.0000 | 0.0062 ± 0.0140 | 0.0178 ± 0.0228 | 0.0000 ± 0.0000 |




**(a) Kosarak** **(b) MovieLens** **(c) Zipf-65K** **(d) Zipf-131K** **(e) UserBehavior**

**Figure 18: Results of E17: Performance (F1) of BSS-HHI and PEMSET under increasing ID permutation ratios with $\varepsilon = 2$ and $k = 32$.**

**Table 4: Results of E18: Singular-value concentration under descending-score ordering with $k = 32$ and $\varepsilon \in \{1, 4\}$.**

| Dataset | $\varepsilon$ | Matrix Size | Top-1 Energy | Top-5 Energy | Rank@90% | Effective Rank | Chosen Rank | Energy at Chosen Rank |
|---|---|---|---|---|---|---|---|---|
| Kosarak | 1 | 53 × 53 | 0.9896 | 1.0000 | 1.0 | 1.393 | 1.3 | 0.9972 |
| Kosarak | 4 | 51 × 51 | 0.9662 | 1.0000 | 1.0 | 1.686 | 2.0 | 0.9999 |
| Zipf-65K | 1 | 63 × 63 | 0.9913 | 1.0000 | 1.0 | 1.447 | 1.3 | 0.9964 |
| Zipf-65K | 4 | 55 × 55 | 0.9957 | 1.0000 | 1.0 | 1.374 | 1.1 | 0.9966 |
| Zipf-131K | 1 | 116 × 116 | 0.9812 | 1.0000 | 1.0 | 1.564 | 1.7 | 0.9977 |
| Zipf-131K | 4 | 116 × 116 | 0.9885 | 1.0000 | 1.0 | 1.488 | 1.7 | 0.9978 |
| MovieLens | 1 | 216 × 216 | 0.9828 | 1.0000 | 1.0 | 1.548 | 1.7 | 0.9976 |
| MovieLens | 4 | 208 × 208 | 0.9968 | 1.0000 | 1.0 | 1.314 | 1.0 | 0.9968 |
| UserBehavior | 1 | 107 × 107 | 0.9956 | 1.0000 | 1.0 | 1.346 | 1.1 | 0.9973 |
| UserBehavior | 4 | 98 × 98 | 0.9998 | 1.0000 | 1.0 | 1.144 | 1.0 | 0.9998 |

**Table 5: Results of E18: Effect of random score ordering on singular-value concentration with $\varepsilon = 4$ and $k = 32$.**

| Dataset | Matrix Size | Top-1 Energy | Rank@90% | Effective Rank |
|---|---|---|---|---|
| Kosarak | 51 × 51 | 0.2006 | 16.7 | 32.313 |
| Zipf-65K | 55 × 55 | 0.3766 | 23.7 | 39.732 |
| Zipf-131K | 116 × 116 | 0.1843 | 52.5 | 87.078 |
| MovieLens | 208 × 208 | 0.2105 | 95.4 | 157.686 |
| UserBehavior | 98 × 98 | 0.7085 | 27.8 | 60.746 |